

Unit 5

SYSTEM SOFTWARE

96 items · 563 marks

Syllabus sections in this unit

5.1 Operating Systems

5.2 Language Translators

Unit 5: System Software

1. M/J 26/P11/Q2/a,b,c

- 2 A computer programmer uses language translators. Two types of translator are an interpreter and a compiler.

(a) Explain why language translators are needed when programming.

[2]

(b) Describe **two** benefits of using an interpreter instead of a compiler when developing a program.

1

2

[4]

(c) Identify **one** other type of translator.

[1]

2. M/J 26/P12/Q7/a,b

7 A student is writing a computer program for a school project.

(a) The student uses a computer that comes with an Operating System (OS) and utility software.

(i) Identify **two** tasks the OS performs as part of process management.

1

2

[2]

(ii) Identify **two** tasks the OS performs as part of memory management.

1

2

[2]

(iii) Explain the reasons why defragmentation software improves the performance of a computer.

[3]

(b) The student is using an Integrated Development Environment (IDE).

(i) Explain how the student can use debugging tools in the IDE.

[4]

(ii) The high-level language the student is using can be partially compiled and partially interpreted.

Explain the reasons why some high-level languages are partially compiled and partially interpreted.

[2]

3. M/J 26/P13/Q5/a,b,c

- 5** A student uses a laptop for school work. The laptop contains system software including an Operating System (OS) and utility software.

(a) Hardware management is one key management task that the OS performs.

- (i)** Identify **two** tasks the OS performs as part of hardware management.

1

2

[2]

- (ii)** Identify **two** other key management tasks that the OS performs.

1

2

[2]

(b) A disk formatter is an example of utility software.

- (i)** State the purpose of utility software in a computer.

[1]

- (ii)** Describe the purpose of a disk formatter.

[2]

(c) The student is writing a computer program in a high-level language.

The student imports routines from program libraries into their program.

- (i)** Define the term program library.

[1]

- (ii)** Describe **one** benefit of the student using a Dynamic Link Library (DLL) instead of a non-Dynamic Link Library.

[2]

4. M/J 26/P21/Q2/b

also Unit 12 (1 mark); this unit 1 mark

- 2** A washing machine contains:

- two trays for the washing powder
- a user interface display
- five sensors.

The washing machine control program includes the following variables:

variable	description
MainsOn	TRUE when the mains power is on
WaterOn	TRUE when the water supply to the machine is turned on
DisplayOn	TRUE when the user interface display is active
Tray1	TRUE when it detects washing powder in tray 1
Tray2	TRUE when it detects washing powder in tray 2

- (b) The user will select from three possible wash options.

The pseudocode for the washing machine program contains a procedure `WashCycle()`

```
PROCEDURE WashCycle()
    // declaration of variables has been omitted
    // TimeSoFar is a global variable which changes
    // as the procedure is executed
    Finished ← FALSE
    TimeSoFar ← 0.0
    OUTPUT "Option (1/2/3) "
    INPUT OptionNumber
    CASE OF OptionNumber:
        // WashTime is the total wash time in minutes
        1 : WashTime ← 60
        2 : WashTime ← 90
        3 : WashTime ← 120
    ENDCASE
    WHILE Finished = FALSE
        TimeRemaining ← WashTime - INT(TimeSoFar)
        
    ENDWHILE
ENDPROCEDURE
```

- (i) The programmer uses an Integrated Development Environment (IDE) to develop and run the washing machine program.

State the IDE feature to display the current value for any variable during the execution of the program.

[1]

- (ii) When option 3 is selected, the programmer interrupts execution when the `TimeSoFar` is 17.6 minutes.

State the current value for `TimeRemaining`

[1]

5. M/J 26/P23/Q2/d

also Unit 12 (1 mark); this unit 1 mark

- 2 A control program is being developed for a robot floor vacuum cleaner. The settings for the robot can be changed and include:

- the strength of the suction (numerical values: 0, 1, 2, 3, 4 or 5)
- the total vacuum cleaning time (a whole number of minutes).

The robot has a number of sensors.

The control program includes the variables:

variable	description
MainsOn	TRUE when the mains power is on
DisplayOn	TRUE when the user display is shown
Suction	TRUE when the program detects that the user has selected a non-zero value for the strength of the suction

(d) The pseudocode for the control program contains the procedure `CleanProgram()`

```

PROCEDURE CleanProgram()
// declaration of variables has been omitted

    Finished ← FALSE
    OUTPUT "Total clean time (minutes) "
    INPUT  TotalCleanTime
    OUTPUT "Suction strength (0,1,2,3,4,5) "
    INPUT  SuctionStrength

    WHILE Finished = FALSE
        // TimeSoFar is a global variable which changes
        // as the procedure is executed
        // TimeSoFar is the time the robot has been cleaning (in seconds)

        TimeRemaining ← TotalCleanTime - INT(TimeSoFar/60)

        }
    ENDWHILE

    }
ENDPROCEDURE

```

- (i) When the program is under development and running, the programmer can temporarily stop the execution at a set point in the program.

Identify this feature of the Integrated Development Environment (IDE).

[1]

- (ii) The programmer runs the program.

- The user is prompted and inputs the value 40 (total clean time in minutes).
- The user is prompted and inputs the value 3 (for the suction strength).
- The programmer interrupts its execution; the `TimeSoFar` is shown in a report window as 125 (the cleaning time so far in seconds).

State the current value of `TimeRemaining`

[1]

6. O/N 25/P11/Q5/a,b,c

5 The Operating System (OS) is responsible for hardware and security management in a computer system.

(a) (i) State **two** tasks that are performed by hardware management.

1

2

[2]

(ii) State **two** tasks that are performed by security management.

1

2

[2]

(b) (i) Describe **one** benefit of using library routines when developing software.

[2]

(ii) Describe **one** drawback of using library routines when developing software.

[2]

(c) Identify **two** presentation features found in a typical Integrated Development Environment (IDE).

1

2

[2]

7. O/N 25/P12/Q8/b(ii)

8 A restaurant accesses software and stores data using cloud computing.

(b) The restaurant uses touchscreens as input devices to allow staff to select items from a menu.

(ii) The image files that are displayed on the touchscreen have been corrupted.

Identify **two** examples of utility software that could be used to recover the images.

1

2

[2]

8. O/N 25/P12/Q10/a

10 A programmer is developing a computer program.

(a) Explain how the programmer can first use an interpreter **and** then a compiler to develop the computer program.

Interpreter

Compiler

[4]

9. O/N 25/P13/Q8/a,b,d

- 8** Compilers and interpreters are used to translate programs written in a high-level language into a low-level language.

(a) State **two** disadvantages of using a compiler compared to an interpreter during program development.

1

2

[2]

(b) Explain how a programmer benefits from using program libraries during program development.

[3]

(d) Memory management is one key management task performed by an OS.

Give **three** other key management tasks that are performed by an OS.

1

2

3

[3]

10. O/N 25/P21/Q1/c

- 1** A program is being developed to meet a particular customer requirement.

(c) During the alpha testing stage, an Integrated Development Environment (IDE) is used to help locate an error that has been identified. The IDE report window feature is used to examine the values assigned to variables.

Explain how **two** other IDE features are used together with the report window feature to help locate the error.

[3]

11. M/J 25/P11/Q6/b

- 6** A programmer is buying a new computer.

(b) Both computers have an Operating System (OS).

Describe the purpose of an OS in a computer.

[5]

12. M/J 25/P13/Q3/a

3 A programmer is writing a program in a high-level language.

(a) Complete the description of compilers and interpreters by writing the missing words.

A compiler checks all the code before attempting to translate the program. If any errors are found, they are all reported at the same time and the program does not translate or run. If there are no errors found, the compiler produces which can run without access to the

An interpreter translates one line of code and then runs it, before moving to the next line of code. If the line of code has an error, the interpreter and displays the error. The programmer can correct the error and then the interpreter continues translating from that point.

[4]

13. O/N 24/P11/Q4/a,c,d

4 A student uses a laptop to write a program that is saved as a text file.

(a) The laptop has utility software and an Operating System (OS).

(i) Describe the file management tasks carried out by an OS.

[2]

(ii) Explain the need for back-up software.

[2]

(c) The student used a program library when writing their program.

Explain the benefits to the student of using library files when writing a program.

[3]

(d) The program code is written using an Integrated Development Environment (IDE).

(i) One presentation feature found in a typical IDE is prettyprint.

Identify **and** describe **one other** presentation feature found in a typical IDE.

Feature

Description

[2]

(ii) One debugging feature found in a typical IDE is single stepping.

Identify **and** describe **one other** debugging feature found in a typical IDE.

Feature

Description

[2]

14. O/N 24/P12/Q3/c

3 A student has a computer.

- (c) The computer has an Operating System (OS). One of the key management tasks of the OS is process management.

Describe the process management tasks performed by an OS.

[4]

15. O/N 24/P12/Q4/a,b

4 A program is written in a high-level language by a team of three programmers using an Integrated Development Environment (IDE).

- (a) Describe how the programmers can use the debugging features of a typical IDE during the development of the program.

[4]

- (b) The programmers created a new program library whilst developing the program.

Describe the benefits to the programmers of creating a program library.

[3]

16. O/N 24/P22/Q1/b,c

1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

- (b) The program contains a function `Lookup()`. After investigation, it is found that this is the function that sometimes returns an incorrect value.

An Integrated Development Environment (IDE) is used to help locate the error.

The IDE features of watch window, single stepping and breakpoint will be used.

Explain these features including the order that they will be used in to locate the error in `Lookup()`.

[3]

- (c) To solve the error a programmer decides to create a new module.

The design of the new module has been completed and the module is being coded.

Identify **two** features of an IDE that will help during the coding of this new module.

1

2

[2]

17. M/J 24/P11/Q3/a,b

3 A software developer is writing a computer program.

- (a) The developer uses an interpreter while writing the program code because it is easier for debugging.

Explain **one** reason why it is easier to debug the program code using an interpreter instead of a compiler.

[2]

- (b) The program is ready to be sold to customers.

The developer uses a compiler because it creates an executable file.

Explain the reasons why the need to create an executable file makes the compiler the appropriate choice when the program is complete.

[3]

18. M/J 24/P12/Q6

6 A computer has an Operating System (OS).

Memory management and process management are two OS tasks.

Explain how memory management **and** process management support multi-tasking.

[4]

19. M/J 24/P12/Q8/a,b,c

8 A programmer uses an Integrated Development Environment (IDE) to write a computer program. The IDE has both a compiler and an interpreter as built-in translators.

- (a) The programmer decides to use the compiler when testing the final program.

Describe the benefits of using the compiler during testing.

[2]

- (b) IDEs have many features other than built-in translators.

Complete the table by identifying **one other** common IDE feature that can be used for each purpose. Describe how each feature helps the user during program development.

Each feature must be different. Do **not** give translator as one of your features.

Purpose	IDE feature	Description
for coding		
for presentation		
for debugging		

[6]

(c) The programmer uses program libraries when developing the program.

Describe **two** benefits to the programmer of using program libraries.

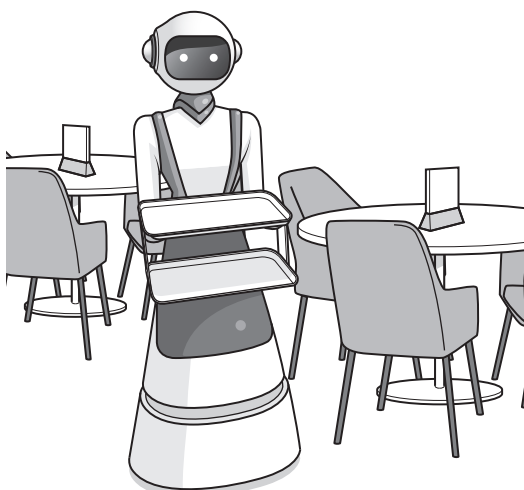
1

2

[2]

20. M/J 24/P13/Q7/e

7 Robots are used to serve food and drink to customers at a restaurant.



(e) Program libraries were used when writing the robot's software.

(i) State what is meant by a **program library**.

[1]

(ii) Some program libraries include Dynamic Link Library (DLL) files.

Describe the benefits of a programmer using a library with DLL files instead of using a library that does not include DLL files.

[4]



21. O/N 23/P11/Q6/a,b,c

- 6** A programmer uses both a compiler and an interpreter to translate a program written in a high-level language.

(a) Describe the advantages of using the interpreter compared to the compiler to translate the program.

[4]

(b) State **one** reason why some high-level languages are partially compiled and partially interpreted.

[1]

(c) (i) Identify **two** features that support the visual presentation of the code in a typical Integrated Development Environment (IDE).

1

2

[2]

(ii) Identify **two** features that support the debugging of the code in a typical IDE.

1

2

[2]

22. O/N 23/P12/Q8/a,b

- 8** A computer has an Operating System (OS).

(a) State **one** purpose of the Operating System.

[1]

(b) The Operating System has utility software.

The purpose of some utility software is to improve security.

Identify **one** example of utility software that is **not** intended to improve security.

Explain why this software is needed.

Utility software

Explanation

[3]

23. O/N 23/P13/Q5/a,b(ii)

- 5 A programmer uses an Integrated Development Environment (IDE) to develop a program that monitors air quality.

(a) Describe the following features of a typical IDE.

Context-sensitive prompts

Single stepping

[4]

(b) The program is distributed by downloading the source code and its library files from a web server.

(ii) The program files are stored on a new hard disk after they have been downloaded.

Describe the reasons why a hard disk formatter is needed for the new hard disk.

[3]

24. M/J 23/P11/Q3/a,b,c

- 3 A computer has an Operating System (OS).

(a) Describe how the Operating System manages the peripheral hardware devices of the computer.

[4]

(b) Hardware management is one key management task carried out by the Operating System.

Identify **two other** key management tasks carried out by the Operating System.

1

2

[2]

(c) The Operating System has utility software including defragmentation software.

Explain how defragmentation can improve the performance of the computer.

[3]

25. M/J 23/P12/Q5/d

- 5 A student has purchased a new laptop.

(d) The laptop has systems software.

(i) Describe how the Operating System (OS) manages processes in the computer.

[5]

(ii) Describe the purpose of utility software in a computer.

[2]

26. M/J 23/P12/Q7/a,b,c

7 A software developer is working in a team writing a program for a client.

(a) The developer is writing a new program library to be used by the other team members.

(i) Define the term **program library**.

[2]

(ii) Explain **two** benefits to the developer of choosing to create a Dynamic Link Library (DLL).

1

2

[4]

(b) The development team needs to use a translator whilst writing the program for the client.

Identify whether an interpreter or a compiler would be more appropriate at this stage of the program development.

Justify your choice.

Translator

Justification

[3]

(c) The development team uses an Integrated Development Environment (IDE).

Complete the table by describing the typical features found in an IDE.

Feature	Description
Breakpoints	
Dynamic syntax checks	
Context-sensitive prompts	
Single stepping	

[4]

27. M/J 23/P13/Q5/a

5 A programmer is developing a computer game in a high-level language to sell to the public.

(a) The programmer uses both an interpreter and a compiler at different stages of the development of the program.

(i) Explain the reasons why the programmer uses an interpreter while writing the program code.

[2]

(ii) Explain the reasons why the programmer uses a compiler when the program has been written.

[3]

28. M/J 23/P21/Q1/a

- 1 A programmer has written a program which includes the function `Calculate()`. When the program is run, the function returns an unexpected value.
- (a) Describe how a typical Integrated Development Environment (IDE) could be used to help debug the program to find the errors in the function `Calculate()`.

[4]

29. M/J 23/P22/Q5/c

- 5 A programmer has produced the following pseudocode to output the square root of the numbers from 1 to 10.

Line numbers are for reference only.

```
10 DECLARE Num : REAL
11 Num ← 1.0
...
40 REPEAT
41     CALL DisplaySqrt(Num)
42     Num ← Num + 1.0
43 UNTIL Num > 10
...
50 PROCEDURE DisplaySqrt(BYREF ThisNum : REAL)
51     OUTPUT ThisNum
52     ThisNum ← Sqrt(ThisNum)           // Sqrt returns the square root
53     OUTPUT " has a square root of ", ThisNum
54 ENDPROCEDURE
```

The pseudocode is correctly converted into program code.

Function `Sqrt()` is a library function and contains no errors.

The program code compiles without errors, but the program gives unexpected results. These are caused by a design error.

- (c) Describe how a typical Integrated Development Environment (IDE) could be used to identify this error.

[3]

30. O/N 22/P11/Q7/a,b

7 (a) State **two** benefits to a programmer of using Dynamic Link Library (DLL) files.

1

2

[2]

(b) Memory management is one of the tasks performed by an Operating System (OS).

Describe the ways in which memory management organises and allocates Random Access Memory (RAM).

[2]

31. O/N 22/P12/Q1/a,b

- 1 (a) Draw **one** line from each utility software to its most appropriate purpose.

Utility software	Purpose
virus checker	to reorganise files so they are contiguous
disk formatter	to scan for malicious program code
backup	to decrease the file size
disk repair	to initialise a disk
defragmentation	to create copies of files in case the original is lost
	to check for and fix inconsistencies on a disk

[5]

- (b) Compilers and interpreters translate programs written in a high-level language into a low-level language.

- (i) State **two** drawbacks of using a compiler compared to an interpreter during program development.

1

2

[2]

- (ii) Explain why high-level language programs might be partially compiled and partially interpreted.

[2]

32. O/N 22/P13/Q3

- 3 Draw **one** line from each Operating System (OS) management task to its most appropriate description.

OS Management task	Description
hardware management	dynamically allocates memory to processes
security management	marks unallocated file storage for availability
memory management	installs programs for devices connected to external ports
process management	validates user and process authenticity
	allows processes to transfer data to and from each other

[4]

33. O/N 21/P11/Q4/b(ii),d

4 Francis is starting his first job as a software developer for a multinational company.

(b) Francis is shown the software he will be working on. He is unfamiliar with the Integrated Development Environment (IDE) he is required to use.

(ii) A typical IDE provides debugging tools to support the testing of a program.

Identify **three** other tools or features found in a typical IDE to support the writing of the program.

1

2

3

[3]

(d) Francis's team use language translators.

Complete the descriptions of language translators by writing the missing words.

..... are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program.

They produce files that can be run without the source code.

..... translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line.

Assemblers are used to translate assembly code into

[4]

34. O/N 21/P12/Q7/b,c

7 A computer has system software.

(b) Describe the file management tasks that an Operating System performs.

[4]

(c) Identify **two** utility programs that can be used to improve the performance of a computer **and** state how they improve the performance.

1

2

[4]

35. O/N 21/P13/Q4/b(ii),d

4 Francis is starting his first job as a software developer for a multinational company.

(b) Francis is shown the software he will be working on. He is unfamiliar with the Integrated Development Environment (IDE) he is required to use.

(ii) A typical IDE provides debugging tools to support the testing of a program.

Identify **three** other tools or features found in a typical IDE to support the writing of the program.

1

2

3

[3]

(d) Francis's team use language translators.

Complete the descriptions of language translators by writing the missing words.

..... are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program.

They produce files that can be run without the source code.

..... translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line.

Assemblers are used to translate assembly code into

[4]

36. 9608 O/N 21/P11/Q7/a,b,c

7 (a) A computer has an operating system (OS).

(i) State **three** memory management tasks performed by the OS.

1

2

3

[3]

(ii) State **two** input/output device management tasks performed by the OS.

1

2

[2]

(b) Utility programs are also provided with the OS.

Identify **and** describe **two** different utility programs.

Utility program

Description

Utility program

Description

[4]

(c) Some operating systems include library programs.

State **three** benefits to a programmer of using library programs.

1

2

3

[3]

37. 9608 O/N 21/P11/Q10/a,b

10 Janaka is developing a new computer program. She decides to use an interpreter instead of a compiler.

(a) State **three** benefits of using an interpreter.

1

2

3

[3]

(b) State **one** drawback of using an interpreter.

[1]

38. M/J 21/P11/Q2/a,b

2 Bingwen's computer comes with an Operating System and utility software.

(a) Draw **one** line from each utility software to its correct description.

Utility software	Description
	Scans software for errors and repairs the problems
Disk formatter	Moves parts of files so that each file is contiguous in memory
Defragmentation	Creates a copy of data that is no longer required
Back-up	Sets up a disk so it is ready to store files
Disk repair	Scans for errors in a disk and corrects them
	Creates a copy of data in case the original is lost

[4]

(b) Identify **four** key management tasks that the Operating System will perform.

1

2

3

4

[4]

39. M/J 21/P12/Q7/a,b

7 Jennifer is writing a computer program for her A Level homework.

- (a)** Jennifer uses a program library to help her write her computer program.

Describe how a program library can be used while writing a computer program.

[2]

- (b)** Jennifer uses an Integrated Development Environment (IDE) to write her computer program.

- (i)** The IDE allows Jennifer to use both an interpreter and a compiler while creating her computer program.

Describe the ways in which Jennifer can use **both** a compiler **and** an interpreter while developing the program.

[4]

- (ii)** Identify **two** debugging tools that a typical IDE can provide.

1

2

[2]

40. M/J 21/P13/Q2/a,b

2 Bingwen's computer comes with an Operating System and utility software.

(a) Draw **one** line from each utility software to its correct description.

Utility software	Description
	Scans software for errors and repairs the problems
Disk formatter	Moves parts of files so that each file is contiguous in memory
Defragmentation	Creates a copy of data that is no longer required
Back-up	Sets up a disk so it is ready to store files
Disk repair	Scans for errors in a disk and corrects them
	Creates a copy of data in case the original is lost

[4]

(b) Identify **four** key management tasks that the Operating System will perform.

1

2

3

4

[4]

41. 9608 M/J 21/P11/Q8/a

8 Jay is developing a computer game that allows users to create stories.

(a) Jay uses a language translator to develop the computer game.

(i) Tick (✓) **one or more** boxes in each row to identify the language translator(s) each statement describes.

Statement	Assembler	Interpreter	Compiler
Translates and executes each line of source code one line at a time			
Translates low-level source code into machine code			
Must be present in memory to execute the code			
Translates high-level source code into low-level code			

[4]

(ii) Jay decides to use a compiler to develop the game.

Identify **two** benefits of using a compiler.

Benefit 1

Benefit 2

[2]

42. 9608 M/J 21/P12/Q7/a

7 Sam is a photographer. She has an image library of over 10 000 images. She stores the images on a high capacity magnetic hard disk.

(a) Explain why Sam would use the following utility software.

(i) Backup

[2]

(ii) Defragmenter

[2]

(iii) Disk repair

[2]

43. 9608 M/J 21/P13/Q4/a(i,iii),b

4 An operating system (OS) is installed on a computer.

(a) The OS performs a number of different tasks such as device management and error detection and recovery.

(i) State **three** device management tasks the OS performs.

1

2

3

[3]

(iii) State **two** tasks, other than device management and error detection and recovery management that are carried out by an OS.

1

2

[2]

(b) Utility programs are installed on a new computer.

(i) The following table lists six programs.

Tick (✓) **one** box in each row to identify whether the program is a utility program or not a utility program.

Program	Utility	Not utility
Language translator		
Backup		
Integrated Development Environment (IDE)		
Graphics		
Defragmenter		
Spreadsheet		

[2]

(ii) Identify **two other** utility programs.

1

2

[2]

44. 9608 M/J 21/P13/Q7/a

- 7 Patrick is writing a new software application. He is using a compiler to develop the software application.

(a) Describe the drawbacks of using a compiler instead of an interpreter.

[3]

45. 9608 M/J 21/P21/Q4/b

- 4 The following is part of a program written in pseudocode:

```
DECLARE ThisArray : ARRAY[1:1000] OF STRING
DECLARE ArrayResult : INTEGER
```

```
PROCEDURE ScanArray(SearchString : STRING)

    DECLARE Index, Total : INTEGER
    DECLARE Error : BOOLEAN

    Index ← 1
    Total ← 0
    Error ← FALSE

    WHILE Index <= 1000 AND Error <> TRUE
        IF LENGTH(ThisArray[Index]) > 5
            THEN
                IF ThisArray[Index] = SearchString
                    THEN
                        Total ← Total + LENGTH(ThisArray[Index])
                    ENDIF
                Index ← Index + 1
            ELSE
                Error ← TRUE
            ENDIF
        ENDWHILE

    ArrayResult ← INT(Total / (Index - 1))

ENDPROCEDURE
```

The procedure `ScanArray()` is amended as follows:

- `SearchString` is compared with just the first **four** characters of each array element.
- The total ignores the first **five** characters of each array element.
- When calculating `ArrayResult`, prevent any possible division by zero.

Refer to the **Appendix** on pages 22–23 for the list of built-in pseudocode functions and operators.

- (b) Context-sensitive prompts are a feature of a typical Integrated Development Environment (IDE).

Explain the term **context-sensitive prompt**.

[2]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
If ThisChar is not an upper case alphabetic character, it is returned unchanged.

Example: LCASE('W') returns 'w'

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

RAND(x : INTEGER) RETURNS REAL
returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.43

NUM_TO_STRING(*x* : REAL) RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if *x* is of type INTEGER
Example: NUM_TO_STRING(87.5) returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

46. 9608 M/J 21/P21/Q5/b(iii)

- 5 Alice is developing an algorithm for a game. The game will count the number of attempts by a player to guess the secret number generated by the built-in random number function.

She will use pseudocode to develop a procedure, `GuessNum()`, that will:

1. generate a secret number that is a random integer in the range 1 to 20 inclusive
2. prompt and input the player's guess
3. output a message if the guess is not the same as the secret number and repeat from step 2
4. count the number of attempts it takes to guess the secret number
5. output a message when the secret number has been guessed correctly, giving the number of attempts.

- (b) Alice is converting her pseudocode into a high-level language for use in a larger modular program.

She wants to start testing the program before all the subroutines (procedures and functions) have been implemented.

- (iii) Alice compiles her program.

Explain the function of the compiler.

47. 9608 M/J 21/P22/Q5/a

5 (a) An Integrated Development Environment (IDE) will be used to develop a program.

(i) An IDE includes features for program presentation.

State **two** of these presentation features.

Feature 1

Feature 2

[2]

(ii) Name **two** IDE features that can help with initial error detection.

Feature 1

Feature 2

[2]

48. M/J 20/P11/Q8/a,b

8 Bart plays computer games on his stand-alone games console.

The games console has an operating system.

(a) Describe the tasks performed by the operating system to manage the **main memory** in the games console.

[4]

(b) The computer games are written in a high-level language. Bart does not need a compiler or an interpreter to run the games he buys for his console.

Explain why the games run without the need for a compiler or an interpreter.

[2]

49. M/J 20/P11/Q9/a

9 Utility programs are examples of system software.

(a) Complete the table by writing the name of the utility program for each description.

Description	Utility program
Reorganises files on a disk to improve efficiency	
Scans a hard disk to identify bad sectors	
Prepares a hard disk for first use	

[3]

50. M/J 20/P12/Q2/c(i,ii)

2 Amir has created a sound file using his desktop computer.

(c) Amir's computer has system software, including utility software and an operating system.

(i) Explain how the disk formatter, disk contents analysis and disk repair utilities work together.

[3]

(ii) Amir's computer has several peripheral devices connected to it.

State **three peripheral** management tasks performed by the operating system.

Task 1

Task 2

Task 3

[3]

51. M/J 20/P13/Q4/a,b

4 Annchi is writing a computer game with a group of friends.

(a) One of her friends has suggested using Dynamic Link Library (DLL) files to help them develop the game.

(i) Give **three** reasons why Annchi and her friends should use DLL files when developing the game.

1

2

3

[3]

(ii) Give **two** reasons why Annchi and her friends should **not** use DLL files when developing the game.

1

2

[2]

(b) Each member of the group is creating a different part of the game. Each person needs to test their part of the game independently before they are combined.

Identify the **most appropriate** type of translator that should be used to test each part of the game independently. Justify your choice.

Translator

Justification

[3]

52. M/J 20/P22/Q1/d

(d) Name **three** features provided by an Integrated Development Environment (IDE) that assist in the coding and **initial** error detection stages of the program development cycle.

1

2

3

[3]

53. M/J 20/P23/Q1/d

(d) Name **two** features provided by a typical Integrated Development Environment (IDE) that assist in the **debugging** stage of the program development cycle.

1

2

[2]

54. M/J 20/P23/Q5/d(iii)

5 A company hires out rowing boats on a lake. The company has 20 boats, numbered from 1 to 20.

For safety reasons, the boats have to be serviced (checked and any damage repaired) regularly.

The company is developing a program to help manage the servicing of the boats.

Every time a boat is serviced, details are added at the end of the text file, `ServiceLog.txt`, as a single line of information. Each boat is serviced before it is hired out for the first time.

The format of each line is as follows:

`<BoatNumber><Date>`

`BoatNumber` and `Date` are as follows:

- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `Date` is an 8-digit numeric string in the format `YYYYMMDD`

The programmer has defined the first module as follows:

Module	Description
<code>GetLastService()</code>	• Called with a string parameter representing the <code>BoatNumber</code> • Searches through the file <code>ServiceLog.txt</code> • Returns the date of the last service in the form "YYYYMMDD"

(iii) Explain how single stepping may be used to help find a logic error in a program.

[2]

55. O/N 19/P11/Q2/b,c,d

2 Aaron uses a desktop computer to do school work.

- (b)** Aaron's computer has an operating system (OS). The OS manages the running processes and provides a user interface.

Describe these OS management tasks.

Process management

Provision of a user interface

[6]

- (c)** Aaron's computer has a virus checker and backup software.

Describe these utility programs.

Virus checker

Backup software

[4]

- (d)** Aaron creates a web page using JavaScript code and HTML tags.

Describe how the JavaScript code is translated using an interpreter.

[2]

56. O/N 19/P12/Q1/a,b

- 1 (a) The diagram shows different types of software on the left, and descriptions on the right.

Draw a line from each type of software to its correct description.

Type of software	Description
Operating system	Provides a ready-built routine that can be imported into a program
Utility program	Provides an interface between the user and the hardware
Library program	Converts source code into a low-level language
Compiler	Creates a new document for the user to edit
	An additional program that helps to maintain or configure the system

[4]

- (b) Describe the purpose of disk repair software.

[3]

57. O/N 19/P12/Q5/b

- 5 Mica has created some software and has copyrighted it. She wants to stop other people from copying and changing it illegally.

- (b) Identify **one** way Mica can distribute the software without the source code.

[1]

58. O/N 19/P13/Q2/a

2 Leonardo's mobile phone has an operating system (OS).

- (a)** Describe the following key management tasks that the mobile phone operating system carries out.

Process management

Memory management

[6]

59. O/N 19/P13/Q6/b

6 Willow is creating a website.

One of the web pages includes the following JavaScript code and HTML tags.

```
01 <html>
02 <body>
03 <form>
04   <p id="displayQuestion"></p>
05   <input type="text" id="answer">
06   <button id="submit" onclick="checkAnswer()"> Submit </button>
07 </form>
08 </body>
09 <script>
10   val1 = Math.floor((Math.random() * 10) + 1);
11   val2 = Math.floor((Math.random() * 10) + 1);
12   document.getElementById("displayQuestion").innerHTML = "What is" +
    val1.toString() + "*" + val2.toString() + "?";
13   function checkAnswer(){
14     userAnswer = document.getElementById("answer").value;
15     answer = val1 * val2
16     if (userAnswer == answer.toString()){
17       alert("Correct, well done");
18     }else{
19       alert("Sorry that's incorrect");
20     }
21   }
22 </script>
23 </html>
```

- (b)** Willow used functions from a JavaScript program library in the web page.

Describe the benefits to Willow of using program libraries to create the web page.

[4]

60. O/N 19/P21/Q2/b

- (b) Identify and describe **one** feature of an Integrated Development Environment (IDE) that can help with **program presentation**.

Feature

Description

[2]

61. O/N 19/P22/Q2/b

- (b) Give **three** features of an Integrated Development Environment (IDE) that can help with **initial error detection** while writing the program.

1

2

3

[3]

62. O/N 19/P23/Q2/c

- (c) A program written in a high-level language cannot be run directly.

Identify **one** type of translator that can be used to translate the program.

[1]

63. M/J 19/P11/Q3/a,b

3 Kimmy has written a program in a high-level language.

(a) Kimmy has used library routines in the program.

(i) Describe **two** advantages of using library routines in the program.

1

2

[4]

(ii) Describe what is meant by a **Dynamic Link Library** (DLL).

[2]

(b) Three translators are compilers, interpreters, and assemblers.

(i) State **one** benefit of Kimmy using an **interpreter** during the development of the program.

[1]

(ii) State **three** benefits of Kimmy using a **compiler** when the program is complete.

1

2

3

[3]

64. M/J 19/P12/Q2/a,b

2 Biyu is writing a computer program in a high-level language.

(a) Biyu uses a language translator.

(i) State the purpose of a language translator.

[1]

(ii) Biyu uses an interpreter.

State **two** benefits of Biyu using an interpreter instead of a compiler while writing the program.

1

2

[2]

(iii) Name a translator other than an interpreter and a compiler.

[1]

(b) Biyu uses library files in the program.

Explain why software is often developed using library files.

[2]

65. M/J 19/P13/Q1/a,b

1 A computer has an operating system (OS) and utility software.

- (a)** The following table lists key management tasks performed by an operating system and their descriptions.

Complete the table by writing the missing management task names and descriptions.

Management task	Description
Memory management	
	Provides user accounts and passwords
	Handles the signals sent when the attention of the processor is required elsewhere
Provision of a software platform	

[4]

- (b)** A hard disk formatter and a hard disk defragmenter are two examples of utility software.

- (i)** Describe the actions performed by a hard disk formatter and a hard disk defragmenter.

Hard disk formatter

Hard disk defragmenter

[4]

- (ii)** Identify **three other** examples of utility software that can be installed on the computer.

1

2

3

[3]

66. M/J 19/P13/Q2/e

- 2 Frankie is a software developer. He is developing a program to manage customer records for a client with an online retail business. He must ensure that data stored about each customer are both secure and private.

(e) When the program is complete, Frankie uses a compiler to prepare the program for the client.

Explain why Frankie uses a compiler instead of an interpreter.

[3]

67. M/J 19/P22/Q2/b

- (b) The program development cycle involves writing, translating and testing a high-level language program.

Describe these activities with reference to **each** of the following:

- editor
- translator
- debugger

[3]

68. M/J 19/P23/Q2/d

- (d) The student has completed the design of her program and is ready to use an Integrated Development Environment (IDE).

Describe the features of an IDE that she can use to write, translate and test her program.

[3]

69. O/N 18/P12/Q6/b,c

6 Kim is using her laptop computer to write a program in a high-level language.

(b) Kim will use library routines in her program.

(i) Describe what is meant by a **library routine**.

[2]

(ii) Describe **one** benefit and **one** drawback of using library routines.

Benefit

Drawback

[4]

(c) Kim develops her program and makes it ready for use. To do this, she uses first an interpreter and then a compiler.

Explain why Kim needs to use both an interpreter and a compiler.

Interpreter

Compiler

[4]

70. M/J 18/P11/Q3/a,b

3 An operating system (OS) is usually pre-installed on a new computer.

(a) The OS performs a number of different tasks such as memory management and security management.

(i) State **three** memory management tasks the OS performs.

1

2

3

[3]

(ii) State **three** security management tasks the OS performs.

1

2

3

[3]

- (iii) State **two** tasks, other than memory management and security management that are carried out by an OS.

1

2

[2]

- (b) Utility software is usually pre-installed on a new computer.

The following table lists four programs. Put **one** tick (✓) in each row to indicate whether or not the program is utility software.

Program	True	False
Disk Defragmenter		
Word Processor		
Library Program		
Compression Software		

[4]

71. M/J 18/P12/Q1/a,b

1 An operating system (OS) is usually pre-installed on a new computer.

(a) The OS performs a number of different tasks such as file management and peripheral management.

(i) State **three** file management tasks the OS performs.

1

2

3

[3]

(ii) State **three** printer management tasks the OS performs.

1

2

3

[3]

(b) Utility software is usually pre-installed on a new computer.

(i) The following table lists four programs. Put **one** tick (✓) in each row to indicate whether or not the program is utility software.

Program	True	False
Database		
Virus checker		
Web browser		
Backup software		

[4]

(ii) Name **two** other utility programs.

Program 1

Program 2

[2]

72. M/J 18/P21/Q5/b

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

(b) Editing functions such as cut, copy and paste are features provided by an Integrated Development Environment (IDE).

Give **two** additional features of an IDE that are helpful when **coding** a program.

Feature 1

Feature 2

[2]

73. M/J 18/P22/Q5/b

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

A program is to be written to store and process the score information. The information to be stored is formed into a string as follows:

`<MembershipNumber><Date><Score>`

(b) A procedure, `AddNewScores()` is being developed. The procedure will be coded using an Integrated Development Environment (IDE).

Name **two** features provided by an IDE that assist in the **initial detection of errors** in the procedure.

Feature 1

Feature 2

[2]

74. M/J 18/P23/Q5/b

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

A program is to be written to store and process the score information.

The information to be stored is as follows:

- `MembershipNumber` is a four-digit numeric string.
- `Date` is a six-digit numeric string in the format DDMMYY
- `Score` is a two-digit numeric string in the range “50” to “99”.

This information is stored in a single string in the format:

`<MembershipNumber><Date><Score>`

- (b) The program will be developed using an Integrated Development Environment (IDE). One feature provided by an IDE is known as `prettyprint`.

Name **two** presentation features provided by `prettyprint`.

Feature 1

Feature 2

[2]

75. O/N 17/P12/Q1

- 1 One of the tasks of the operating system (OS) is the management of the main memory of the computer system.

State and describe **three** other operating system management tasks.

1

2

3

[6]

76. O/N 17/P12/Q2/a,b

- 2 (a)** The diagram shows three items of software that translate program code.

Draw **one** line from each context to the correct item of translation software.

Context	Item of translation software
The source code is written in a high-level language. An executable file is produced.	Assembler
The source code uses instructions from the processor's instruction set.	Interpreter
The source code and translation software must both be in main memory at execution time.	Compiler
A web page contains some JavaScript code.	

[4]

- (b)** A programmer is developing software and has both a compiler and interpreter for the high-level language used.

Describe **two** benefits of using each form of translation software.

- (i)** Benefits of a compiler

1

2

[2]

- (ii)** Benefits of an interpreter

1

2

[2]

77. O/N 17/P13/Q2/a,b

- 2 (a) The diagram shows three items of software that translate program code.

Draw **one** line from each context to the correct item of translation software.

Context	Item of translation software
A web page contains a client-side script.	Assembler
Each instruction in the source code consists of an op code and an operand.	Interpreter
The source code is required at run-time.	Compiler
When the source code is translated, copies of the executable program can be distributed without the need for the source code.	

[4]

- (b) The Java programming language is said to be machine or platform independent.

- (i) Describe what is meant by **machine independent**.

[1]

- (ii) Describe how a Java source code program is translated.

[2]

78. O/N 17/P22/Q4/b

- 4 Programming languages provide built-in functions to generate random numbers. To be truly random, the frequency of each number generated should be the same.

You are required to write program code to test the random number generator of your chosen language.

The test should:

- generate a given number of random numbers between 1 and 10 inclusive
- keep a count of the number of times each number is generated
- calculate the expected frequency of each number 1 to 10
- output the actual frequency of each number 1 to 10
- output the difference between the actual frequency and the expected frequency.

The program code should be written as a procedure. In pseudocode, the procedure heading will be:

```
PROCEDURE TestRandom(Repetitions AS INTEGER)
```

The parameter, `Repetitions`, contains a value representing the total number of random numbers that should be generated.

The following example shows the expected output for the procedure call, `TestRandom(200)`.

The expected frequency is 20.

Number	Frequency	Difference
1	17	-3
2	21	1
3	12	-8
4	28	8
5	20	0
6	19	-1
7	21	1
8	16	-4
9	24	4
10	22	2

- (a) Write **program code** for the procedure, `TestRandom`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[16]

- (b) Name **three** features of a typical IDE that would help a programmer to debug a program.

Explain how each of these could be used in the debugging of the `TestRandom` procedure from **part (a)**.

Feature 1

Explanation

Feature 2

Explanation

Feature 3

Explanation

[6]

79. M/J 17/P11/Q6/a,b

- 6 (a)** The operating system (OS) contains code for performing various management tasks.

The appropriate code is run when the user performs various actions.

Draw a line to link each OS management task to the appropriate user action.

OS management task	Action
Main memory management	The user moves the mouse on the desktop
Input/Output management	The user closes the spreadsheet program
Secondary storage management	The user selects the Save command to save their spreadsheet file
Human computer interface management	The user selects the Print command to output their spreadsheet document

[3]

- (b)** A user has the following issues with the use of his PC.

State the utility software which should provide a solution.

- (i)** The hard disk stores a large number of video files. The computer frequently runs out of storage space.

Utility software solution [1]

- (ii)** The user is unable to find an important document. He thinks it was deleted in error some weeks ago. This must not happen again.

Utility software solution [1]

- (iii)** The operating system reports 'Bad sector' errors.

Utility software solution [1]

- (iv)** There have been some unexplained images and advertisements appearing on the screen. The user suspects it is malware.

Utility software solution [1]

80. M/J 17/P12/Q4/a,b

- 4 (a) (i) Explain why a personal computer (PC) needs an operating system (OS).

[2]

- (ii) One of the tasks carried out by the OS is the management of the use of the processor.

Name and describe **two** other management tasks that the OS performs.

1

2

[4]

- (b) A user has the following issues with the use of their personal computer (PC).

For each case, state the utility software which should provide a solution.

- (i) The user wants to send a large file as an attachment to an email. The user knows that the recipient's Internet Service Provider (ISP) has a limit of 2MB for file attachments.

Utility software solution: [1]

- (ii) The user is writing a book and is worried that the document files could get damaged or deleted.

Utility software solution: [1]

- (iii) The computer has recently been slow to load large files. The user has deleted a large number of small files to try to solve the problem. A friend has advised that there is a procedure which should be regularly carried out to reorganise file storage on the hard disk.

Utility software solution: [1]

- (iv) The user clicked on an attachment in an unsolicited email. Since then, the computer has shown some unexplained behaviours.

Utility software solution: [1]

81. M/J 17/P13/Q6/a,b

- 6 (a)** The operating system (OS) contains code for performing various management tasks.

The appropriate code is run when the user performs various actions.

Draw a line to link each OS management task to the appropriate user action.

OS management task**Action**

Main memory management	The user moves the mouse on the desktop
Input/Output management	The user closes the spreadsheet program
Secondary storage management	The user selects the Save command to save their spreadsheet file
Human computer interface management	The user selects the Print command to output their spreadsheet document

[3]

- (b)** A user has the following issues with the use of his PC.

State the utility software which should provide a solution.

- (i)** The hard disk stores a large number of video files. The computer frequently runs out of storage space.

Utility software solution

[1]

- (ii)** The user is unable to find an important document. He thinks it was deleted in error some weeks ago. This must not happen again.

Utility software solution

[1]

- (iii)** The operating system reports 'Bad sector' errors.

Utility software solution

[1]

- (iv)** There have been some unexplained images and advertisements appearing on the screen. The user suspects it is malware.

Utility software solution

[1]

82. O/N 16/P11/Q2/b,c

(b) (i) Explain why a computer needs an operating system.

[2]

(ii) Give **two** key management tasks carried out by an operating system.

1

2

[2]

(c) New program code is to be written in a high-level language. The use of Dynamic Link Library (DLL) files is considered in the design.

Describe what is meant by a DLL file.

[2]

83. O/N 16/P11/Q7/a

7 A small company produces scientific magazines. The owner buys some new desktop computers. The computers are used to store thousands of colour images (diagrams and photographs). All the computers have Internet access.

(a) Name **three** utility programs the company would use on all their computers. Describe what each program does.

1

Description

2

Description

3

Description

[6]

84. O/N 16/P12/Q7

7 One management task carried out by an operating system is to provide a user interface.

Describe **two** more of these management tasks.

1

2

[4]

85. O/N 16/P12/Q8/a,b

8 A programmer is writing a program that includes code from a program library.

(a) Describe **two** benefits to the programmer of using one or more library routines.

1

2

[4]

(b) The programmer decides to use a Dynamic Link Library (DLL) file.

(i) Describe **two** benefits of using DLL files.

1

2

[4]

(ii) State **one** drawback of using DLL files.

[2]

86. O/N 16/P13/Q2/b,c

(b) (i) Explain why a computer needs an operating system.

[2]

(ii) Give **two** key management tasks carried out by an operating system.

1

2

[2]

(c) New program code is to be written in a high-level language. The use of Dynamic Link Library (DLL) files is considered in the design.

Describe what is meant by a DLL file.

[2]

87. O/N 16/P13/Q7/a

- 7** A small company produces scientific magazines. The owner buys some new desktop computers. The computers are used to store thousands of colour images (diagrams and photographs). All the computers have Internet access.

(a) Name **three** utility programs the company would use on all their computers. Describe what each program does.

1

Description

2

Description

3

Description

[6]

88. O/N 16/P21/Q4/a,b

- 4** A company employs Ahmed as a programmer.

(a) At College, before joining the company, Ahmed used two items of software for programming:

- a text editor
- a compiler

Describe how he could have developed programs using these software tools.

Include in the description the terms 'object code' and 'source code'.

[3]

(b) Ahmed now uses an Integrated Development Environment (IDE) for programming.

(i) State **one** feature an IDE provides to help with the identification of syntax errors.

[1]

(ii) State **one** feature an IDE provides to carry out white box testing.

[1]

89. O/N 16/P23/Q4/a,b

4 A company employs Ahmed as a programmer.

(a) At College, before joining the company, Ahmed used two items of software for programming:

- a text editor
- a compiler

Describe how he could have developed programs using these software tools.

Include in the description the terms 'object code' and 'source code'.

[3]

(b) Ahmed now uses an Integrated Development Environment (IDE) for programming.

(i) State **one** feature an IDE provides to help with the identification of syntax errors.

[1]

(ii) State **one** feature an IDE provides to carry out white box testing.

[1]

90. M/J 16/P11/Q1

1 Three examples of language translators and four definitions are shown below.

Draw lines to link each language translator to the correct one or more definitions.

Language translator**Definition**

Compiler

The software reads the source code and reports all errors. The software produces an executable file.

Assembler

The software reads each statement and checks it before running it. The software halts when it encounters a syntax error.

Interpreter

The software translates a high-level language program into machine code for the processor to execute.

The software translates low-level statements into machine code for the processor to execute.

[3]

91. M/J 16/P12/Q1

- 1 Three examples of language translators and four definitions are shown below.

Draw lines to link each language translator to the correct one or more definitions.

Language translator**Definition**

Compiler

The software reads the source code and reports all errors. The software produces an executable file.

Assembler

The software reads each statement and checks it before running it. The software halts when it encounters a syntax error.

Interpreter

The software translates a high-level language program into machine code for the processor to execute.

The software translates low-level statements into machine code for the processor to execute.

[3]

92. M/J 16/P13/Q1

- 1 Describe **two** differences between a compiler and interpreter.

1

2

[4]

93. O/N 15/P11/Q11

- 11** A game program is written which can be either interpreted or compiled. The table below shows five statements about the use of interpreters and compilers.

Tick (✓) to show whether the statement refers to an interpreter or to a compiler.

Statement	Interpreter	Compiler
This translator creates an executable file		
When this translator encounters a syntax error, game execution halts		
The translator analyses and checks each line just before executing it		
This translator will produce faster execution of the game program		
Use of this translator makes it more difficult for the user to modify the code of the game		

[5]

94. O/N 15/P13/Q11

- 11** A game program is written which can be either interpreted or compiled. The table below shows five statements about the use of interpreters and compilers.

Tick (✓) to show whether the statement refers to an interpreter or to a compiler.

Statement	Interpreter	Compiler
This translator creates an executable file		
When this translator encounters a syntax error, game execution halts		
The translator analyses and checks each line just before executing it		
This translator will produce faster execution of the game program		
Use of this translator makes it more difficult for the user to modify the code of the game		

[5]

95. O/N 15/P21/Q2

2 A programmer uses an Integrated Development Environment (IDE) for all program development.

(i) Describe what is meant by an IDE.

[2]

(ii) Name **three** features you would expect to be available in an IDE to help initial error detection or debugging.

1

2

3

[3]

96. O/N 15/P23/Q2

2 A programmer uses an Integrated Development Environment (IDE) for all program development.

(i) Describe what is meant by an IDE.

[2]

(ii) Name **three** features you would expect to be available in an IDE to help initial error detection or debugging.

1

2

3

[3]

Answers Section

1. M/J 26/P11/Q2/a,b,c

2(a)	1 mark each to max 2 - To convert a high-level language to a low-level language/assembly/machine code - Computers use machine code and programmers use high-level languages - To allow the computer to process the coded statements	2	Accept human language for HLL in MP2 MP3: Read is NE
2(b)	1 mark for each benefit and 1 mark for an appropriate expansion to max 4 (2 × 2) For example: - Amend the code or debug in real time - and see the changes immediately instead of having to recompile - Stops as soon as an error is encountered - so the position of the error can be easily seen - Only one error is displayed at a time // no dependent errors generated - individual errors are easier to identify/debug/fix - Incomplete code can be tested/run - so the program does not need to be complete to see if it works/see the output	4	
2(c)	1 mark for a correct answer Assembler	1	Correct answer only

2. M/J 26/P12/Q7/a,b

7(a)(i)	1 mark for each correct point to max 2 - Efficient allocation of resources to processes - Moving processes between states - Allocating time slices to processes - Scheduling processes // An example of a scheduling algorithm - Resolution of conflicts when two processes require the same resource - Allows communication between processes	2	Do not accept allows multi-programming – that is a benefit not a task resource management is NE Accept checking priorities as scheduling Take care that answers do not refer to tasks done by the kernel or by memory management.
7(a)(ii)	1 mark for each correct point to max 2 - Dynamically allocates RAM to processes - Frees up memory when the process closes - Makes sure two processes do not access the same memory - Accesses data from RAM when needed	2	Accept software or applications for processes in first mark point Take care that answers do not refer to tasks done by process management or by file management
7(a)(iii)	1 mark for each correct point to max 3 - With continued use files can be split and stored in different locations on the disk so time is spent by the disk head moving between locations to access all parts of file - Defragmentation software moves split files into contiguous blocks // defragmentation software collects all the free space into a contiguous block ... which means that the disk head does not need to move as much to access the next part of the file // ... which means that the disk head does not need to move as much when writing a file - After defragmentation data access/read/write takes less time, so performance is improved	3	Note: The question asks how defragmentation improves computer performance. So just a description of the process is NE

7(b)(i)	1 mark for each correct point to max 4 , mark in pairs • (Single) Stepping • ...the student runs one line of code at a time and looks at the output/results/flow before moving to run the next line • Breakpoints • ...These are inserted in the code to stop the code at a specific point so that the student can check the flow/values / test sub-sections of the code • (Variable) Watch window • ...the student can view how/when the variables change during execution	4	Take care that the answer says how the student uses the tools and doesn't just describe each tool Award a maximum of two marks for names of debugging tools with no further suitable explanation Just check for errors is not enough for an explanation, that is what debugging means
7(b)(ii)	1 mark for each correct point to max 2 • Partially compiled programs can be used on different platforms as they are interpreted when run • The code is optimised for the CPU as machine code is generated at run time • It is more efficient to run as the source code does not need recompiling	2	Note: Candidate responses need to answer the question why?

3. M/J 26/P13/Q5/a,b,c

5(a)(i)	1 mark each to max 2 e.g. • Transfer data between peripheral devices • Translate data from I/O/peripheral devices • Loading and installation of device drivers • Hardware abstraction — hiding hardware differences from applications • Power management — sleep states, CPU throttling, battery control	2	
5(a)(ii)	1 mark each to max 2 e.g. • Memory management • Process management • File management • Security management // managing user accounts • Provide platform for running applications	2	
5(b)(i)	1 mark for: • Perform housekeeping/maintenance tasks // improving/optimising overall performance	1	
5(b)(ii)	1 mark each to max 2 e.g. • Prepared a disk for future use by // initialising • ...changing/setting file system // by example // directory structure // allocation tables • ... naming the disk (volume label) • ... creating a boot sector • ...erasing all data • ...creating partitions	2	
5(c)(i)	1 mark for: • Set of pre-written / pre-compiled / pre-tested routines	1	

5(c)(ii)	1 mark for benefit 1 for expansion e.g. • Maintenance not needed to be done by the programmer • ... because the DLL is separate from program // maintained by library creator • The calling program does not need recompilation by the programmer when a DLL file changes • ... because the DLL file can be updated independently of the calling program • ... updates will apply to all programs that use the DLL file • (Executable) file is smaller • ... because it does not contain (all) the library routines // code only linked at execution • (Main) memory requirements for program is reduced • ... because the DLL files are only loaded into memory when required • ... one DLL file can be used by several running programs	2	
----------	--	----------	--

4. M/J 26/P21/Q2/b

2(b)(i)	Report window // Watch window	1
2(b)(ii)	103	1

5. M/J 26/P23/Q2/d

2(d)(i)	<u>breakpoint</u>	1	
2(d)(ii)	38 (minutes)	1	

6. O/N 25/P11/Q5/a,b,c

5(a)(i)	1 mark per bullet point, max 2 marks • Installs <u>driver</u> software for devices connected to external ports • Manages communication between devices • Manages <u>hardware</u> interrupts	2
5(a)(ii)	1 mark per bullet point, max 2 marks • Prevents unauthorised access • ... by providing authentication // by validating users and processes • Implements access rights and permissions • Makes provision for recovery of lost data • Carries out Operating System security updates as available • Carries out auditing and keeps logs of activity	2

5(b)(i)	One mark for benefit, one mark for appropriate matching expansion e.g. • Less code needs to be developed as code is pre-written • ... thereby reducing the time to write a program • The library routine will be pre-tested and documented • ... reducing the time taken to test a program // giving confidence that the routine works as it should • The code for the library routine can be written in a different programming language • ... which allows the use of special features of that language • The user's source code is more readable • ... as the library routine is simply called in the source code rather than being written out in full • The library could contain the code for complex routines • ... thus allowing the programmer to include modules they might not be able to code	2
5(b)(ii)	1 mark for drawback, 1 mark for appropriate matching expansion e.g. • Compatibility issues • ... the library routine may not work with current program • Not guaranteed to be thoroughly tested • ... there could be unexpected problems (bugs/virus) • The library routine may not match needs exactly // The library routine may give unexpected results • ... so it will need to be edited and tested • If the library routine is later edited or changed (by someone else) • ... there could be unexpected errors or results	2
5(c)	1 mark per bullet point, max 2 marks • Prettyprint • Expanding and collapsing code blocks • <u>Automatic</u> / <u>smart</u> indentation / formatting	2

7. O/N 25/P12/Q8/b(ii)

8(b)(ii)	1 mark per bullet point, max 2 marks • Disk repair / disk contents analysis • Back-up software	2
----------	--	----------

8. O/N 25/P12/Q10/a

10(a)	1 mark per bullet point, max 4 marks Max 2 from each section. Interpreter: • Use an interpreter while writing/coding the program • ... to test / debug the partially completed program • ... because errors can be corrected and processing continue from where the execution stopped // errors can be corrected in real time // errors are identified one at a time Compiler: • Use the compiler after the program is complete • ... to create an executable file so source code not seen • Use the compiler to repeatedly test the same completed section without having to re-interpret every time	4
-------	---	----------

9. O/N 25/P13/Q8/a,b,d

8(a)	1 mark per bullet point, max 2 • Large amounts of source code take time to compile • It can be slower to produce the object code than an interpreter • The code must be recompiled when it is changed • The program cannot run if there are errors • It is not possible to correct errors in real-time • One error can cause false reporting of multiple further errors • Sections of code / unfinished code cannot easily be tested	2
8(b)	1 mark per bullet point, max 3 marks • Programming time is saved as code does not have to be written from scratch • Testing time is saved as code is already tested / documented • A library routine is more likely to work, as code is already tested • Library routines automatically update if they are changed / improved • The programmer can use library routines to perform complex functions / procedures that they may not be able to write themselves	3
8(d)	1 mark for each bullet point, max 3 marks • File management • Security management • Hardware / peripheral management // input / output management • Process management	3

10. O/N 25/P21/Q1/c

1(c)	One mark per point: 1 reference to a breakpoint 2 reference to single stepping 3 Correct explanation of both e.g. to stop program execution at specific point / line / statement / instruction and to execute one line / statement / instruction at a time to (locate an/the error) One mark for each bulleted point	3
------	---	----------

11. M/J 25/P11/Q6/b

6(b)	1 mark each to max 5 e.g. • To provide a user interface • so that the user is able to communicate with the hardware • To manage memory • so that data can be stored and accessed • and multitasking is possible • To manage files • allowing the user to create, edit, update and delete files and folders • To manage inputs and outputs from hardware/peripherals • To handle processes • to make sure each process has fair access	5
------	--	----------

12. M/J 25/P13/Q3/a

3(a)	1 mark for each correctly completed term • an executable file/.exe • source/program code • stops • immediately/in real-time A compiler checks all of the code before attempting to translate the program. If any errors are found, they are all reported at the same time and the program does not translate or run. If there are no errors found, the compiler produces an executable file/.exe which can run without access to the source/program code. An interpreter translates one line of code and then runs it, before moving to the next line of code. If the line of code has an error, the interpreter stops and displays the error. The programmer can correct the error immediately/in real-time and then the interpreter continues translating from that point.	4
------	---	----------

13. O/N 24/P11/Q4/a,c,d

4(a)(i)	1 mark for each bullet point (max 2) e.g. • Secondary storage space is divided into file allocation units • Space on secondary storage is allocated to particular files • Maintains/creates directory structures • Specifies the logical method of file storage e.g. FAT or NTFS • Provides file naming conventions • Controls user access // implements access rights //implements password protection // Makes file sharing possible • Controls access to the file by other software • Specifies tasks that can be performed on a file e.g. open, close, delete, copy, create, move, etc • Allows searching for a file	2
4(a)(ii)	1 mark for each bullet point (max 2) • To allow data to be retrieved / restored when lost • To automatically make a duplicate copy of data... • ...so the user does not have to remember to backup data • To make regular duplicate copies of data	2
4(c)	1 mark for each bullet point (max 3) e.g. • It saves development time for the student • ... by using pre-written code • There is less testing to do // the routines are more likely to work correctly • ... because they have already been tested and are error-free • It results in a more robust program • Subroutines can be used that they do not have the skill to write • ... which enables a more complex program to be written	3
4(d)(i)	1 mark for correct name, 1 mark for matching description • Expand/collapse code blocks... • ... sections of source code that are part of the same block can be expanded to see the content or collapsed so that the overall code is seen • Auto-indentation // auto-formatting... • ... automatically indents/formats code as the user types so that the structure is clear // aids readability	2
4(d)(ii)	1 mark for correct name, 1 mark for matching description • Breakpoints ... • ... stops the code running on a set line to view the current status/variable contents/program flow • Report window // variable watch window ... • ... shows the values in variables/data structures and how they change when each line is run	2

14. O/N 24/P12/Q3/c

3(c)	1 mark for each bullet point (max 4) e.g. • It manages the scheduling of processes // Decides which process is to be run next • Allows multi-tasking/multi-processing • Ensures fair access • Handles interrupts • Manages / allocates which resources the processes require • Facilitates the sharing and exchange of data between processes • Prevents interference between processes // conflict resolution	4
------	---	----------

15. O/N 24/P12/Q4/a,b

4(a)	1 mark for each bullet point (max 4) e.g. • Single stepping • Run the program one line at a time • ... and check the variable contents / program flow // show the effect of each line of code • Set breakpoints • ... run the code up to a set line • ... and then check the status • Variable/report watch window • ... view how the data changes as the program is running	4
4(b)	1 mark for each bullet point (max 3) e.g. • Subroutines can be shared / reused • ... between team members who are working independently • ... without having to rewrite/re-test them which saves the programmers' time • A program library provides continuity between programs/programmers • Individual programmers can contribute their specialisms to the library // Individual programmers can use the specialisms of others	3

16. O/N 24/P22/Q1/b,c

1(b)	For example: • Set a breakpoint at the start of / within <code>Lookup</code>, to stop execution at a given statement • then use single stepping to execute one statement / instruction at a time • to display the value of variables using a watch window One mark for each: MP1 Order starting with a breakpoint and an explanation – ‘stop execution at this statement / line’ MP2 Explanation of single stepping – execute ‘line by line’ / statements MP3 Explanation of watch window – displaying the value of <u>variable(s)</u>	3
1(c)	Features include: MP1 Editor MP2 <u>Auto</u>- (syntax) complete / <u>auto</u> correction // identify undeclared variables(s) MP3 Prettyprint / <u>auto</u>-indentation / <u>auto</u> (structure) highlighter MP4 <u>Dynamic</u> syntax checking MP5 Expand / collapse code blocks MP6 Context sensitive prompts Max 2 marks	2

17. M/J 24/P11/Q3/a,b

3(a)	1 mark each to max 2: • The interpreter will stop when an error is found • ... so the error can be corrected in real-time, and the result of changes seen immediately • Only one error is displayed at a time • ... so fewer errors to correct simultaneously and no dependent errors	2
3(b)	1 mark each to max 3: • Program can be distributed without source code • ... so it cannot be edited/stolen/plagiarised • Users do not require the translator to run the program • ... so time is not spent retranslating by user	3

18. M/J 24/P12/Q6

6	1 mark each to max 4 Max 3 marks for each management task: <i>Memory management: Max 3 marks</i> • Stores data from all currently running programs concurrently in RAM • Stops the data from overwriting each other in RAM/primary storage • Decides which processes should be in main memory • Makes efficient use of memory <i>Process management: Max 3 marks</i> • Allows one process to be paused whilst another process can be actioned • Decides which process is to be run next • Switches between processes to allow them to share the use of the processor • Identification/description of scheduling	4
---	---	----------

19. M/J 24/P12/Q8/a,b,c

8(a)	1 mark each to max 2: • Creates an executable file • ... so the code can be tested multiple times without having to recompile • ... so repeated testing takes less time	2
------	--	----------

8(b)	1 mark for identification of each feature and 1 mark for matching description: e.g. <i>For coding:</i> <table><tr><th>IDE feature</th><th>Description</th></tr><tr><td>Context-sensitive prompts</td><td>Gives suggestions for code as the user types instead of having to write/remember the code</td></tr><tr><td>Auto-correct</td><td>Corrects spelling mistakes so that user has fewer errors to correct</td></tr></table> <i>For presentation:</i> <table><tr><th>IDE feature</th><th>Description</th></tr><tr><td>Pretty-printing</td><td>Colour code keywords so the user can identify any errors</td></tr><tr><td>Expand/collapse (code) blocks</td><td>The user can hide code that they are not currently working on</td></tr></table> <i>For debugging:</i> <table><tr><th>IDE feature</th><th>Description</th></tr><tr><td>Single stepping</td><td>Run the code one line at a time // shows the effect of each line of code</td></tr><tr><td>Breakpoints</td><td>Stop the code running at a set point to check the flow/variable contents</td></tr></table>	IDE feature	Description	Context-sensitive prompts	Gives suggestions for code as the user types instead of having to write/remember the code	Auto-correct	Corrects spelling mistakes so that user has fewer errors to correct	IDE feature	Description	Pretty-printing	Colour code keywords so the user can identify any errors	Expand/collapse (code) blocks	The user can hide code that they are not currently working on	IDE feature	Description	Single stepping	Run the code one line at a time // shows the effect of each line of code	Breakpoints	Stop the code running at a set point to check the flow/variable contents	6
IDE feature	Description																			
Context-sensitive prompts	Gives suggestions for code as the user types instead of having to write/remember the code																			
Auto-correct	Corrects spelling mistakes so that user has fewer errors to correct																			
IDE feature	Description																			
Pretty-printing	Colour code keywords so the user can identify any errors																			
Expand/collapse (code) blocks	The user can hide code that they are not currently working on																			
IDE feature	Description																			
Single stepping	Run the code one line at a time // shows the effect of each line of code																			
Breakpoints	Stop the code running at a set point to check the flow/variable contents																			
8(c)	1 mark each to max 2: • Saves programming/testing time as code does not have to be written/re-written from scratch // code does not have to be tested • Code is already tested so it is more robust/likely to work • The programmer does not need to maintain the library // library routines are updated automatically • Can perform complex calculations that the programmer may be unable to do • Makes code more easily readable	2																		

20. M/J 24/P13/Q7/e

7(e)(i)	1 mark for: Pre-written code/functions/routines that can be imported/called in another program	1
---------	--	----------

7(e)(ii)	1 mark each to max 4 : • Maintenance not needed to be done by the programmer • ... because the DLL is separate from program • The calling program does not need recompilation by the programmer when a DLL file changes • ... because the DLL file can be updated independently of the calling program • ... updates will apply to all programs that use the DLL file	4
----------	---	----------

21. O/N 23/P11/Q6/a,b,c

6(a)	1 mark for each bullet point (max 4) • easier to debug the program • ... because it translates line-by-line and stops when an error is found whereas the compiler translates all the program at the same time • ... only reporting one error at a time • ... which allows the error to be corrected in real time whereas the program would need to be corrected and recompiled • ... and the program can restart at same point when error occurred with a compiler the program needs to be re-run • The effect of any changes made by the programmer can be seen immediately with a compiler the effects can only be seen after re-running • A partially completed program can be translated / tested on its own a compiler cannot translate a partial program	4
6(b)	1 mark for each bullet point (max 1) • Partially compiled programs can be used on different platforms as they are interpreted when run • Code is optimised for the CPU as machine code is generated at run time • Source code does not need recompiling so more efficient to run	1
6(c)(i)	1 mark for each bullet point (max 2) • Prettyprint • Expand/collapse code blocks • Auto indentation / formatting	2
6(c)(ii)	1 mark for each bullet point (max 2) • Single stepping • Breakpoints • Report window • Variable expressions	2

22. O/N 23/P12/Q8/a,b

8(a)	1 mark for each bullet point (max 1) - To hide the complexities of the hardware from the user - To provide a platform for software to run - To provide a user interface	1
8(b)	1 mark for the name of the utility software 2 marks for the explanation - Defragmentation software ... because over time saving and deleting of small files fragments the disk ... the software makes (individual) files contiguous ... so access time to the files is improved ... because head movement is reduced - Disk contents analysis/disk repair software ... to identify and mark bad sectors ... to restore corrupted files ... to recover lost data (due to hardware failure) - File compression ... to reduce the size of files ... which saves storage and memory space // by example ... and reduces transmission time // by example - Disk formatter ... to prepare a disk for use // set up the file system ... to partition the disc ... to delete all the data from the disc	3

23. O/N 23/P13/Q5/a,b(ii)

5(a)	1 mark for each bullet point (max 2 for each feature). Context-sensitive prompts: - As the code is being written ...the options to complete the statement are shown Single stepping: - allows the programmer to execute the program one line at a time ...so that the effects of each statement can be seen	4
5(b)(ii)	1 mark for each bullet point (max 3). - Disk needs to be prepared for initial use - Disk needs to be checked for errors - A new file system needs to be generated on the disk - The file allocation table needs to be set up	3

24. M/J 23/P11/Q3/a,b,c

3(a)	1 mark each to max 4 Examples: • Installs device drivers • ... to allow communication between peripherals and computer • Sends data and receives data to and from peripherals • ... such as to an output device and from an input device/by example • Handles buffers for transfer of data • ... to ensure smooth transfer between devices that transmit and receive at different speeds • Manages interrupts / signals from the device	4
3(b)	1 mark each to max 2 • Memory management • File management • Security management • Process management • Error checking and recovery	2
3(c)	1 mark each to max 3 • Rearranges blocks of individual files (on the HDD) so they are contiguous // moves the free space together • Accessing each file is faster • ...because there is no need to search for the next fragment / block of the file • ...so less head movement is needed	3

25. M/J 23/P12/Q5/d

5(d)(i)	1 mark each to max 5 • Manages the scheduling of processes // decides which order to run processes • Manages which resources the processes require • ... such as allocating memory • Enables processes to share data • Prevents interference between processes // resolution of conflicts • Handles the process queue • It allows multi-tasking / multi-processing • ... by ensuring fair access, handling priorities and handling interrupts	5
5(d)(ii)	1 mark each to max 2 • To help users to set-up / configure / analyse / optimise / maintain the computer ... • ... by for example, making memory allocation more efficient • ... by for example, checking the system for faults	2

26. M/J 23/P12/Q7/a,b,c

7(a)(i)	1 mark each to max 2 • Set of pre-written / pre-compiled / pre-tested subroutines • ... which can be called in other programs • ... by installing/importing the library	2
7(a)(ii)	1 mark for each bullet point. Mark in pairs; 1 mark for a benefit and 1 mark for an appropriate expansion • (main) memory requirements for program is reduced • ... as dynamic link library is loaded only once / when required • the executable file size of the program using the DLL will be smaller • ... because the executable does not contain (all) the library routines • maintenance not needed to be done by the programmer • ... because the DLL is separate from program • no need to recompile the main program when changes are made to DLL • ... because changes / improvements/ error correction to the DLL file code are done independently of the main program • A single DLL file can be made available to several application programs • ... Saving space in memory / easing the pressure on memory	4
7(b)	No mark for choice. 1 mark each to max 3 for justification Interpreter • Allows the developer to make real-time changes • ... so the program can be debugged at each stage • ... the effect of any changes made by the developer can be seen immediately • The developer can test when incomplete • ... so small parts can be tested without having to test the rest of the program • ... if one section does not work others can still be tested • To avoid dependent errors Compiler • The developer can debug multiple errors simultaneously • Produces an executable file • ... so that the developer can test the program multiple times without recompiling	3

7(c)	1 mark each Breakpoints: • Stop the code at a specific line to check the current progress / values Dynamic syntax checks: • Highlight / underline / colour syntax errors as the code is entered Context-sensitive prompts: • Suggest the code to add // automatically complete statements Single stepping: • Run the code one line at a time so the values can be checked	4
------	--	----------

27. M/J 23/P13/Q5/a

5(a)(i)	1 mark each to max 2 • Programmer can test sections of the code without every part working / being written • Programmer can debug in real time • ... so that errors can be fixed and the program continued from that point • The effect of any changes made by the programmer can be seen immediately • To avoid dependent errors	2
5(a)(ii)	1 mark each to max 3 • The compiler produces an executable file • ... so the user cannot access / edit / sell the code • ... and users do not need the translator to run the game • The game can be compiled for different hardware specifications • ... and then used to generate more income for the programmer • The program can be tested multiple times without having to retranslate each time	3

28. M/J 23/P21/Q1/a

1(a)	For example: Could set a <u>breakpoint</u> to stop the program at a particular point / instruction then the value of variables could be checked using a <u>report/watch window</u> while <u>single stepping</u> can be used to execute one statement/line at a time. Marks available as follows: • One mark for each underlined term • One mark for an explanation of each term Note: max 4 marks	4
------	---	----------

29. M/J 23/P22/Q5/c

5(c)	Could use an IDE to: • Set a breakpoint to stop the program at a certain line / statement / point • Step through the program line by line / statement by statement • checking the value of 'num' / a variable using a report / watch window One mark per bullet	3
------	---	----------

30. O/N 22/P11/Q7/a,b

7(a)	1 mark for each benefit (max 2): • (main) memory requirements for program are reduced as dynamic link library is loaded only once / when required • the executable file size is smaller because the executable does not contain all the library routines • maintenance not needed to be done by the programmer because the DLL is separate from program • no need to recompile the main program when changes are made to DLL because changes / improvements/ error correction to the DLL file code are done independently of the main program	2
7(b)	1 mark for each bullet point (max 2): • RAM is assigned into blocks • dynamic allocation of RAM to programs / processes • reclaims unused blocks of RAM • prevents two programs / processes occupying the same area of RAM at the same time • moves data from secondary storage when needed // manages paging, segmentation and virtual memory	2

31. O/N 22/P12/Q1/a,b

1(a)	1 mark for each correct line. <table><thead><tr><th>Utility software</th><th>Purpose</th></tr></thead><tbody><tr><td>virus checker</td><td>to reorganise files so they are contiguous</td></tr><tr><td>disk formatter</td><td>to scan for malicious program code</td></tr><tr><td>backup</td><td>to decrease the file size</td></tr><tr><td>disk repair</td><td>to initialise a disk</td></tr><tr><td>defragmentation</td><td>to create copies of files in case the original is lost</td></tr><tr><td></td><td>to check for and fix inconsistencies on a disk</td></tr></tbody></table>	Utility software	Purpose	virus checker	to reorganise files so they are contiguous	disk formatter	to scan for malicious program code	backup	to decrease the file size	disk repair	to initialise a disk	defragmentation	to create copies of files in case the original is lost		to check for and fix inconsistencies on a disk	5
Utility software	Purpose															
virus checker	to reorganise files so they are contiguous															
disk formatter	to scan for malicious program code															
backup	to decrease the file size															
disk repair	to initialise a disk															
defragmentation	to create copies of files in case the original is lost															
	to check for and fix inconsistencies on a disk															
1(b)(i)	1 mark for each bullet point (max 2): • larger amounts of source code take time to compile • slower to produce the object code than an interpreter • code cannot be changed without recompilation • the program will not run if there are any errors • errors cannot be corrected in real-time • one error may result in other false errors being reported • cannot easily test specific sections of the source code // cannot easily test unfinished source code	2														
1(b)(ii)	1 mark for each bullet point (max 2): • partially compiled programs can be used on different platforms as they are interpreted when run • code is optimised for the CPU as machine code is generated at run time	2														

32. O/N 22/P13/Q3

3	1 mark for each correct line: <table><tr><th>OS Management task</th><th>Description</th></tr><tr><td>hardware management</td><td>dynamically allocates memory to processes</td></tr><tr><td>security management</td><td>marks unallocated file storage for availability</td></tr><tr><td>memory management</td><td>installs programs for devices connected to external ports</td></tr><tr><td>process management</td><td>validates user and process authenticity</td></tr><tr><td></td><td>allows processes to transfer data to and from each other</td></tr></table>	OS Management task	Description	hardware management	dynamically allocates memory to processes	security management	marks unallocated file storage for availability	memory management	installs programs for devices connected to external ports	process management	validates user and process authenticity		allows processes to transfer data to and from each other	4
OS Management task	Description													
hardware management	dynamically allocates memory to processes													
security management	marks unallocated file storage for availability													
memory management	installs programs for devices connected to external ports													
process management	validates user and process authenticity													
	allows processes to transfer data to and from each other													

33. O/N 21/P11/Q4/b(ii),d

4(b)(ii)	1 mark for each correct tool e.g. • Colour coding // pretty printing • Auto-complete • Auto-correct • Context sensitive prompts • Expand and collapse code blocks	3
4(d)	1 mark for each correctly completed term Compilers are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program. They produce executable/.exe/object code files that can be run without the source code. Interpreters translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line. Assemblers are used to translate assembly code into binary/machine code.	4

34. O/N 21/P12/Q7/b,c

7(b)	1 mark for each bullet point to max 4 - Storage space is divided into file allocation units - Space is allocated to particular files - Maintains / creates directory structures - Specifies the logical method of file storage (e.g. FAT or NTFS) - Provides file naming conventions - Controls access // implements access rights // implements password protection // Makes file sharing possible - Specifies tasks that can be performed on a file (e.g. open, close, delete, copy, create, move etc.)	4
7(c)	1 mark for identifying program 1 mark for description, max 2 per program e.g. - Defragmentation - Less time is taken to access files because each one is contiguous so there is less head movement - Virus checker - makes more RAM available for programs to run ... because it removes software that might be taking up memory / replicating - Disk repair / Disk contents analysis - preventing bad sectors being used because it identifies / marks them - reduces access times by optimising storage - Disk/system clean up - releases storage by removing unwanted / temporary files	4

35. O/N 21/P13/Q4/b(ii),d

4(b)(ii)	1 mark for each correct tool e.g. - Colour coding // pretty printing - Auto-complete - Auto-correct - Context sensitive prompts - Expand and collapse code blocks	3
4(d)	1 mark for each correctly completed term Compilers are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program. They produce executable/.exe/object code files that can be run without the source code. Interpreters translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line. Assemblers are used to translate assembly code into binary/machine code.	4

36. 9608 O/N 21/P11/Q7/a,b,c

7(a)(i)	One mark per bullet point to max 3 • Prevents two processes occupying the same memory space • Organises memory e.g. paging/segmentation • Makes uses of virtual memory • Keeps track of allocated and free memory locations • Allocates memory to processes • Ensures fair use of memory • Releases memory when a process stops	3
7(a)(ii)	One mark per bullet point to max 2 • Installs drivers • Sends/receives data from a buffer // buffer management // • Sends commands to the device • Receives/handles messages/signals/interrupts from the device • Control of hardware usage by processes • Device detection • Power management • Keep track of device status (free or busy)	2
7(b)	One mark for name and one for description × 2, max 4 The description must match the name • Virus/Malware checker • ...scans disk for viruses and reports, quarantines or deletes them • File compression • ...reduces the size of a file • Back up • ...creates a copy of data in case of loss, so that the images can be restored • Defragmenter • ...finds files that are not stored in contiguous blocks and moves them together • Disk formatter • ...creates a logical drive on a hard drive // reformats a previously used hard drive // creates logical partitions // prepares a disk for first use • Disk repair/Disk contents analysis • ...attempts to recover damaged files // checks disk for space and usage // to identify/mark the errors / bad sectors	4

7(c)	One mark per bullet point to max 3 • Already tested • Should be relatively free from errors // more robust • Used by many others • Make use of another programmer's knowledge • Precompiled • Saves (programming/testing) time	3
------	--	----------

37. 9608 O/N 21/P11/Q10/a,b

10(a)	One mark per bullet point to max 3 • Easier de-bugging • Errors can be corrected in real time • Errors are reported as the interpreter finds them. • An error can be corrected and translation continued from where it stopped • The effect of any change made to the code can be seen immediately • Parts of the program can be tested, without all the program code being available.	3
10(b)	One mark for a correct answer Source code is needed at run time // No executable file produced, (so source code can be edited) // Translation software needed every time the program is run // execution time increased	1

38. M/J 21/P11/Q2/a,b

2(a)	1 mark for each correct line <table><thead><tr><th>Utility software</th><th>Description</th></tr></thead><tbody><tr><td>Disk formatter</td><td>Scans software for errors and repairs the problems</td></tr><tr><td>Defragmentation</td><td>Moves parts of files so that each file is contiguous in memory</td></tr><tr><td>Back-up</td><td>Creates a copy of data that is no longer required</td></tr><tr><td>Disk repair</td><td>Sets up a disk so it is ready to store files</td></tr><tr><td></td><td>Scans for errors in a disk and corrects them</td></tr><tr><td></td><td>Creates a copy of data in case the original is lost</td></tr></tbody></table>	Utility software	Description	Disk formatter	Scans software for errors and repairs the problems	Defragmentation	Moves parts of files so that each file is contiguous in memory	Back-up	Creates a copy of data that is no longer required	Disk repair	Sets up a disk so it is ready to store files		Scans for errors in a disk and corrects them		Creates a copy of data in case the original is lost	4
Utility software	Description															
Disk formatter	Scans software for errors and repairs the problems															
Defragmentation	Moves parts of files so that each file is contiguous in memory															
Back-up	Creates a copy of data that is no longer required															
Disk repair	Sets up a disk so it is ready to store files															
	Scans for errors in a disk and corrects them															
	Creates a copy of data in case the original is lost															
2(b)	1 mark per bullet point to max 4 • memory management • file management • security management • hardware / device / peripheral / resources management • input/output management • process management • error checking and recovery • provision of a platform for software • provision of a user interface	4														

39. M/J 21/P12/Q7/a,b

7(a)	1 mark per bullet point to max 2 • Program libraries store pre-written functions and routines • The program library can be referenced/imported • the functions/routines can be called in her own program	2
------	--	----------

7(b)(i)	1 mark per bullet point to max 4; max 3 from each section Interpreter: • Use an interpreter while writing the program • ... to test/debug the partially completed program • ... because errors can be corrected and processing continue from where the execution stopped // errors can be corrected in real time // errors are identified one at a time Compiler: • Use the compiler after the program is complete • ... to create an executable file • Use the compiler to repeatedly test the same (completed) section • ... without having to re-interpret every time // compiler not needed at run-time	4
7(b)(ii)	1 mark per correct tool to max 2 e.g. • Breakpoints • Single stepping • Report windows	2

40. M/J 21/P13/Q2/a,b

2(a)	1 mark for each correct line <table><tr><th>Utility software</th><th>Description</th></tr><tr><td>Disk formatter</td><td>Scans software for errors and repairs the problems</td></tr><tr><td>Defragmentation</td><td>Moves parts of files so that each file is contiguous in memory</td></tr><tr><td>Back-up</td><td>Creates a copy of data that is no longer required</td></tr><tr><td>Disk repair</td><td>Sets up a disk so it is ready to store files</td></tr><tr><td></td><td>Scans for errors in a disk and corrects them</td></tr><tr><td></td><td>Creates a copy of data in case the original is lost</td></tr></table>	Utility software	Description	Disk formatter	Scans software for errors and repairs the problems	Defragmentation	Moves parts of files so that each file is contiguous in memory	Back-up	Creates a copy of data that is no longer required	Disk repair	Sets up a disk so it is ready to store files		Scans for errors in a disk and corrects them		Creates a copy of data in case the original is lost	4
Utility software	Description															
Disk formatter	Scans software for errors and repairs the problems															
Defragmentation	Moves parts of files so that each file is contiguous in memory															
Back-up	Creates a copy of data that is no longer required															
Disk repair	Sets up a disk so it is ready to store files															
	Scans for errors in a disk and corrects them															
	Creates a copy of data in case the original is lost															

2(b)	1 mark per bullet point to max 4 - memory management - file management - security management - hardware / device / peripheral / resources management - input/output management - process management - error checking and recovery - provision of a platform for software - provision of a user interface	4
------	--	----------

41. 9608 M/J 21/P11/Q8/a

8(a)(i)	1 mark per correct row	4																				
<table><tr><th>Statement</th><th>Assembler</th><th>Interpreter</th><th>Compiler</th></tr><tr><td>Translates and executes each line of source code one line at a time</td><td></td><td>✓</td><td></td></tr><tr><td>Translates low-level source code into machine code</td><td>✓</td><td></td><td></td></tr><tr><td>Must be present in memory to execute the code</td><td></td><td>✓</td><td></td></tr><tr><td>Translates high-level source code into low-level code</td><td></td><td>✓</td><td>✓</td></tr></table>			Statement	Assembler	Interpreter	Compiler	Translates and executes each line of source code one line at a time		✓		Translates low-level source code into machine code	✓			Must be present in memory to execute the code		✓		Translates high-level source code into low-level code		✓	✓
Statement	Assembler	Interpreter	Compiler																			
Translates and executes each line of source code one line at a time		✓																				
Translates low-level source code into machine code	✓																					
Must be present in memory to execute the code		✓																				
Translates high-level source code into low-level code		✓	✓																			
8(a)(ii)	1 Mark per bullet point to max 2 • The compiler is not required to run the program • The program can be distributed without the source code • A compiler produces code that executes faster than the equivalent for an interpreter	2																				

42. 9608 M/J 21/P12/Q7/a

7(a)(i)	1 mark per bullet point to max 2 - The images on the hard disk can be lost or corrupted - To make a copy of her files (which can be stored elsewhere) - So that the images can be restored	2
7(a)(ii)	1 mark per bullet point to max 2 - Frequent changes to the images mean the data for each file is split across the disk - To rearrange the (fragmented) files into contiguous locations // to bring all the empty space together - To improve the time it takes to access and load the files	2

7(a)(iii)	1 mark per bullet point to max 2 • Areas of the disk can become corrupt • To identify/mark the errors/bad sectors • ...so they can be repaired // so bad sectors are no longer used	2
-----------	--	----------

43. 9608 M/J 21/P13/Q4/a(i,iii),b

4(a)(i)	1 mark per bullet point to max 3 • Install/manage device drivers • Control of hardware usage by processes // allocation of devices to processes // inter process communication • Device detection • Power Management • Keep track of device status (free or busy) • Buffer management	3
---------	---	----------

4(a)(iii)	1 mark per bullet point to max 2 • Process / task/resource management • Main memory management • File/secondary storage management • Security management • Provision of a software platform/environment on which other programs can be run • Interrupt handling • Provision of a user interface NOT • Peripheral/hardware/device management • Error detection and recovery management	2
-----------	---	----------

4(b)(i)	1 mark for first 3 rows 1 mark for last 3 rows <table border="1"> <thead> <tr> <th>Program</th><th>Utility</th><th>Not utility</th></tr> </thead> <tbody> <tr> <td>Language translator</td><td></td><td>✓</td></tr> <tr> <td>Backup</td><td>✓</td><td></td></tr> <tr> <td>Integrated Development Environment (IDE)</td><td></td><td>✓</td></tr> <tr> <td>Graphics</td><td></td><td>✓</td></tr> <tr> <td>Defragmenter</td><td>✓</td><td></td></tr> <tr> <td>Spreadsheet</td><td></td><td>✓</td></tr> </tbody> </table>	Program	Utility	Not utility	Language translator		✓	Backup	✓		Integrated Development Environment (IDE)		✓	Graphics		✓	Defragmenter	✓		Spreadsheet		✓	2
Program	Utility	Not utility																					
Language translator		✓																					
Backup	✓																						
Integrated Development Environment (IDE)		✓																					
Graphics		✓																					
Defragmenter	✓																						
Spreadsheet		✓																					

4(b)(ii)	1 mark per bullet point to max 2 e.g. • Virus checker // anti-malware • Disk formatter • Disk contents analysis/disk repair • System clean-up • File compression • Firewall • Encryption	2
----------	--	----------

44. 9608 M/J 21/P13/Q7/a

7(a)	1 mark per bullet point to max 3 • The program will not run if there are any errors • The program must be recompiled after every change // cannot correct errors in real-time • Part-programs cannot be tested • One error may result in other false errors being reported	3
------	--	----------

45. 9608 M/J 21/P21/Q4/b

4(b)	One mark for each: • The IDE displays hints / choice of keywords / available identifiers • (Appropriate to) the current cursor position / insertion point	2
------	--	----------

46. 9608 M/J 21/P21/Q5/b(iii)

5(b)(iii)	A compiler is used to translate / convert the source code / program / high-level language code into <u>object code</u> / <u>machine code</u> / <u>an executable file</u>	1
-----------	--	----------

47. 9608 M/J 21/P22/Q5/a

5(a)(i)	One mark per point: • Pretty print / Colour-coding of keywords / variables • (Auto) indentation • (Auto) Expansion / collapsing of data structures / code blocks // thumbnail overview Max 2 marks	2
5(a)(ii)	One mark per point: • Dynamic syntax checking / highlighting syntax errors as code is typed • Type checking • Checking for used variables not declared / unused variables which are declared Max 2 marks	2

48. M/J 20/P11/Q8/a,b

8(a)	1 mark per bullet point to max 4 - Reads/writes data to/from RAM ... e.g. current data/instructions from a game so the CPU can access it - Allocates virtual memory ... when there is insufficient RAM to run a program/game - Allocates RAM to optimise performance - Paging - Segmentation	4
8(b)	1 mark per bullet point to max 2 - Software will have been built using a compiler // the software is pre-compiled - Software is an executable file // the game is already in machine code // the game is already set-up to run on the console - Source code is not provided so does not need compiling/interpreting	2

49. M/J 20/P11/Q9/a

9(a)	1 mark for each correctly identified utility program	3								
<table><tr><th>Description</th><th>Utility program</th></tr><tr><td>Reorganises files on a disk to improve efficiency</td><td>Defragmentation software</td></tr><tr><td>Scans a hard disk to identify bad sectors</td><td>Disk contents analysis / repair software</td></tr><tr><td>Prepares a hard disk for first use</td><td>Disk formatter</td></tr></table>		Description	Utility program	Reorganises files on a disk to improve efficiency	Defragmentation software	Scans a hard disk to identify bad sectors	Disk contents analysis / repair software	Prepares a hard disk for first use	Disk formatter	
Description	Utility program									
Reorganises files on a disk to improve efficiency	Defragmentation software									
Scans a hard disk to identify bad sectors	Disk contents analysis / repair software									
Prepares a hard disk for first use	Disk formatter									

50. M/J 20/P12/Q2/c(i,ii)

2(c)(i)	1 mark per bullet point to max 3 - Disk contents analysis checks for errors/problems with the disk - Disk repair attempts to fix the errors - The disk formatter prepares the disk for (initial) use (again).	3
2(c)(ii)	1 mark per bullet point to max 3 - Installation of device driver software - Managing interrupts / signals from the device - Sending control signals to the device - Control of buffers - Management of queues	3

51. M/J 20/P13/Q4/a,b

4(a)(i)	1 mark per reason to max 3 • DDL file is only loaded into memory when required • ... so the executable file for the game is smaller • Changes/improvements in the DLL file are independent of the main program • ... the game program will not need to be recompiled • ... the game program will get the benefit of the updates automatically • The same DDL file can be used in several game programs (at the same time) • (DLL) routines are pre-written saving the developers time • (DLL) routines are pre-tested so should be reliable • Developers can take advantage of other programmers' expertise	3
4(a)(ii)	1 mark per reason to max 2 • Game will not work if DDL is corrupted • An external change to the DDL could stop the game working or change the way it works • The DDL file must be present at run-time otherwise there is an error	2
4(b)	1 mark for interpreter 1 mark per bullet point for justification to max 2 • The interpreter translates and executes line by line • The interpreter stops translation when an error is found • ...the error can be corrected, and the program continues running from where it stopped • There is no need for an executable file (at this stage)	3

52. M/J 20/P22/Q1/d

1(d)	Examples include: • context sensitive prompts • (dynamic) syntax checking • use of colours to highlight key words / pretty printing / highlighting unused variables (etc) • Formatting (incl. collapsing and expanding blocks) • (UML) modelling • Text editor (or by reference to a function such as copy & paste) • Built-in (library) functions Do not accept answers relating to debugging features Max 3	3
------	--	----------

53. M/J 20/P23/Q1/d

1(d)	• Breakpoints • Single-stepping • Watch window Max 2	2
------	---	----------

54. M/J 20/P23/Q5/d(iii)

5(d)(iii)	One mark per bullet point • Executes a line of code at a time • Used to trace the path of execution (sequence) • Track variable values using a watch window Max 2	2
-----------	---	----------

55. O/N 19/P11/Q2/b,c,d

2(b)	1 mark per bullet point to max 4 for each management task, max 6 in total Process management: • Manages the scheduling of processes • ... allows multi-tasking / multi-processing • ... ensures fair access • ... handles priorities • Manages the resources the processes need • Enables processes to share information • Prevents interference between processes// resolution of conflicts Provision of a user interface: • Allows a user to communicate with the hardware // vice-versa • ... by making navigation around the system easier • Provides facility for user inputting data • Provides facility for outputting to the user • By example e.g. command line / GUI / menu-driven	6
------	--	----------

2(c)	1 mark per bullet point to max 3 for each utility program, max 4 in total Virus checker: • Scans files stored on a computer system for malicious code • Scans files when they enter the system / memory stick inserted / download etc. • Sets up a schedule for virus-checking • Isolates / quarantines / deletes viruses • Regularly updates the virus definitions Backup software: • Creates a copy of the contents of a disk / partition. Can be set up to automatically backup // schedules backups • Allows the user to decide what is backed up, e.g. all data // all files that have changed since the last backup • Allows the user to set up an off-site backup • May encrypt the backup files • Restores the data if necessary	4
2(d)	1 mark per bullet point to max 2 • The code is translated one line at a time • ... and executed immediately • The interpreter stops as soon as it finds an error	2

56. O/N 19/P12/Q1/a,b

1(a)	1 mark per correct line	4												
<table><thead><tr><th>Type of software</th><th>Description</th></tr></thead><tbody><tr><td>Operating system</td><td>Provides a ready-built routine that can be imported into a program</td></tr><tr><td>Utility program</td><td>Provides an interface between the user and the hardware</td></tr><tr><td>Library program</td><td>Converts source code into a low-level language</td></tr><tr><td>Compiler</td><td>Creates a new document for the user to edit</td></tr><tr><td></td><td>An additional program that helps to maintain or configure the system</td></tr></tbody></table>			Type of software	Description	Operating system	Provides a ready-built routine that can be imported into a program	Utility program	Provides an interface between the user and the hardware	Library program	Converts source code into a low-level language	Compiler	Creates a new document for the user to edit		An additional program that helps to maintain or configure the system
Type of software	Description													
Operating system	Provides a ready-built routine that can be imported into a program													
Utility program	Provides an interface between the user and the hardware													
Library program	Converts source code into a low-level language													
Compiler	Creates a new document for the user to edit													
	An additional program that helps to maintain or configure the system													

1(b)	1 mark per bullet point to max 3 Disk repair: • Checks for any errors / inconsistencies / bad sectors on the disk • Resolves any errors on the disk • Retrieves files / data from a damaged disk // re-constructs directory // recovers disc when data corrupt • Marks bad sectors on the disk // marks bad sectors as unusable	3
------	--	----------

57. O/N 19/P12/Q5/b

5(b)	• Provide an .exe file // Compile the source code // Use a compiler	1
------	---	----------

58. O/N 19/P13/Q2/a

2(a)	1 mark per bullet point to max 4 for each management task, max 6 in total Process Management • Manages the scheduling of processes • ... allows multi-tasking / multi-processing • ... ensures fair access • ... handles priorities • Manages which resources the processes require • Enables processes to share information • Prevents interference between processes // resolution of conflicts Memory Management • Allocates memory to processes • Ensures fair usage of memory • Organises memory / by example • Makes use of virtual memory • Keep processes separate • To release memory when a process stops	6
------	---	----------

59. O/N 19/P13/Q6/b

6(b)	1 mark per bullet point to max 4 • Using program libraries saves time • ... as she does not have to write some routines • Program library routines should already be thoroughly tested • ... so, Willow can use them without having to spend time checking they work / correcting errors • Program library routines may perform complex tasks • ... so, Willow won't have to work out how to code them	4
------	--	----------

60. O/N 19/P21/Q2/b

2(b)	One mark for name and one mark for explanation. Example: • PrettyPrint // Colour coding • Colour coding of command words / key words • Expand and collapse code blocks • Allows programmer to focus on a section of code // allows quicker navigation of the code • Auto(matic) indentation • Allows the programmer to clearly see the different code sections / easier to see the code structure Accept suitable alternatives	2
------	--	----------

61. O/N 19/P22/Q2/b

2(b)	Any three from: 1. Dynamic syntax checking // Identification of syntax errors 2. Highlighting undeclared variables // incorrect variable usage... 3. Parameter checking 4. Type checking 5. Auto-indentation 6. PrettyPrint	3
------	--	----------

62. O/N 19/P23/Q2/c

2(c)	Compiler / Interpreter	1
------	------------------------	----------

63. M/J 19/P11/Q3/a,b

3(a)(i)	1 mark for each advantage, 1 mark for a valid expansion to max 2 × 2 • Code is already tested • ...so it is more robust/likely to work • Saves programming time • ...code does not have to be written/re-written from scratch • The programmer can use e.g. mathematical functions • ...that s/he may not know how to code • If there is an improvement in the library routine • ...the program updates automatically	4
---------	--	----------

3(a)(ii)	1 mark per bullet point to max 2 • A collection of self-contained (shared library) programs • ...that are already compiled • Linked to the main program during execution • Library program code is separate from the .EXE file • Library file only loaded into memory when required at run time • A DLL file can be made available to several applications (at the same time) • If DLL routine is updated the program that uses it will run the update	2
3(b)(i)	1 mark per bullet point to max 1 • Errors can be corrected as they occur • Can run a partially complete program when developing • The effect of any change made to the code can be seen immediately	1
3(b)(ii)	1 mark per bullet point to max 3 • Produces an executable file • User does not have access to source code • It will (probably) be faster to run the executable • Code does not have to be compiled each time it is run • Does not need the compiler to be present at run-time	3

64. M/J 19/P12/Q2/a,b

2(a)(i)	1 mark for correct answer • To convert a (higher level) programming language to a different form	1
2(a)(ii)	1 mark per bullet point to max 2 • Easier de-bugging • ...because errors can be corrected in real time • The effect of any change made to the code can be seen immediately • Parts of the program can be <u>tested</u>, without all the program code being available	2
2(a)(iii)	1 mark for correct answer Assembler	1

2(b)	1 mark per bullet point to max 2 - The code is already written so the programmer is not starting over again which saves time - The code will have been used by many people so it should be already thoroughly tested and relatively error-free / won't need re-testing - The programmer can use e.g. mathematical / graphics functions etc. that he may not know how to code - Use of named library functions can simplify the program and make it easier to read - The library routine code should conform to industry standards and therefore contribute towards a more robust program	2
------	---	----------

65. M/J 19/P13/Q1/a,b

1(a)	1 mark for each correct entry <table><tr><th>Management task</th><th>Description</th></tr><tr><td>Memory management</td><td>Handles the allocation of memory to processes // Ensures two programs do not attempt to use the same memory locations // Keeps track of allocated and free memory locations</td></tr><tr><td>Security management</td><td>Provides user accounts and passwords</td></tr><tr><td>Interrupt processing</td><td>Handles the signals sent when the attention of the processor is required elsewhere</td></tr><tr><td>Provision of a software platform</td><td>Provides an environment within which programs can be run</td></tr></table>	Management task	Description	Memory management	Handles the allocation of memory to processes // Ensures two programs do not attempt to use the same memory locations // Keeps track of allocated and free memory locations	Security management	Provides user accounts and passwords	Interrupt processing	Handles the signals sent when the attention of the processor is required elsewhere	Provision of a software platform	Provides an environment within which programs can be run	4
Management task	Description											
Memory management	Handles the allocation of memory to processes // Ensures two programs do not attempt to use the same memory locations // Keeps track of allocated and free memory locations											
Security management	Provides user accounts and passwords											
Interrupt processing	Handles the signals sent when the attention of the processor is required elsewhere											
Provision of a software platform	Provides an environment within which programs can be run											
1(b)(i)	1 mark per bullet point to max 2 for formatter, max 2 for defragmenter hard disk formatter • Makes existing data inaccessible • Partitions the disk into logical drives • Sets up the (specified) file system • Prepares the disk for initial use • May check for errors on the disk hard disk defragmenter • Re-organises the disk contents • Moves split files so they are contiguous • Creates a larger area of (contiguous) free space	4										

1(b)(ii)	1 mark per bullet point For example: • Backup software • File compression • Virus checker • Disk contents analysis / repair	3
----------	--	----------

66. M/J 19/P13/Q2/e

2(e)	1 mark per bullet point to max 3 • He uses a compiler because it creates a separate executable file • The executable means the client cannot access the source code // edit the program • The executable means the client does not need the IDE / compiler • It will (probably) be faster to run the executable than to interpret the source code every time the program is run	3
------	---	----------

67. M/J 19/P22/Q2/b

2(b)	• An editor is used to produce / write / modify the <u>source code</u> / <u>program</u> / <u>high-level language code</u> OR by example: An editor provides (features such as) context-sensitive prompts / dynamic syntax checking etc. • A translator (compiler) is used to translate / convert the source code / program / high-level language code into <u>object code</u> / <u>machine code</u> / <u>an executable file</u>. OR A translator (interpreter) is used to translate the source code / program / high-level language code <u>line by line</u> • A debugger is used to test the program / detect errors (and correct errors) in the program. One mark per bullet point	3
------	---	----------

68. M/J 19/P23/Q2/d

2(d)	Write: Use an editor to write the source code / program / high-level language code. <i>Or by example of feature:</i> An editor provides (features such as) context-sensitive prompts / dynamic syntax checking / PrettyPrint / auto-indentation etc. Translate: A translator (compiler) will convert the source code / program / high-level language code into <u>object code</u> / <u>machine code</u> / <u>an executable file</u> A translator (interpreter) is used to translate the source code / program / high-level language code <u>line by line</u> // Or by example: identify syntax errors <i>Or by example of feature:</i> A translator will identify errors Test: A debugger is used to find / (help to) correct errors. <i>Or by example of feature:</i> e.g. single-step, break-points, watch-window... One mark per category (Write, Translate, Test) for each reference to a specific 'feature'.	3
------	--	----------

69. O/N 18/P12/Q6/b,c

6(b)(i)	1 mark per bullet to max 2 • Pre-existing / pre-compiled / pre-written modules / code ... • ... can be linked into her program (without amendment) • To perform common / complex tasks	2
---------	---	----------

6(b)(ii)	1 mark per bullet point. Max 2 for <u>one benefit</u>, max 2 for <u>one drawback</u> Benefit: • Less code needs to be written • ... saves time / saves re-inventing the wheel • Pre-tested // Used by many people • ... reduces time testing // can be fairly sure that the function will perform as it should • Can be written in a different programming language • ... making use of special features of that language • Can be complex algorithms (e.g. mathematical/graphics functions) • ... she does not need to work out how to write it // ...that she may not know how to code • Simplifies the program • ... since just the name of the function included in the source code Drawback: • Compatibility issues • ... may not work with the other code/may require changing program for it to work • Not guaranteed thorough testing • ... may be unknown or unexpected bugs / virus • Library routine may not meet exact needs • ... may give unexpected results // ... may need editing • If library routine is changed • ... there may be unexpected results / errors	4
6(c)	1 mark per bullet point. Max 3 marks for interpreter, max 3 marks for compiler Interpreter: • Used during development • Debugging is easier • ... Because errors are reported as they are found // No need to wait until the end of the process for the error report • ... Because errors can be corrected as they are found Compiler: • Compiler used when development complete // compiler used when program ready for distribution • Produces an executable file (.exe) • ... After compilation the compiler does not need to be present for the program to run • ... The program can be given to others without access to (source) code • ... Final program does not need to be re-compiled each time it is run • Cross-compilation, the program can be compiled to run on different platforms	4

70. M/J 18/P11/Q3/a,b

3(a)(i)	1 mark per bullet to max 3 • Allocates / deallocates RAM to programs/tasks/processes • Keeps track of allocated and free memory locations • Swaps data to and from the hard drive • Handles virtual memory • Paging // segmentation • Memory protection, preventing a process accessing memory not allocated to it	3															
3(a)(ii)	1 mark per bullet to max 3 • Sets up user accounts • Checks usernames, passwords // Authentication • Implements access rights • <u>Automatic</u> backup • System restore / roll back (to previous stable state)	3															
3(a)(iii)	1 mark per bullet to max 2 • Device / peripheral management • File management • Process management • Input / output management • Error detection / recovery • Provides a user interface • Facilitates communication between hardware and software / hardware devices	2															
3(b)	1 mark for each correct box ticked <table border="1"> <thead> <tr> <th>Programs</th><th>True</th><th>False</th></tr> </thead> <tbody> <tr> <td>Disk Defragmenter</td><td>✓</td><td></td></tr> <tr> <td>Word Processor</td><td></td><td>✓</td></tr> <tr> <td>Library program</td><td></td><td>✓</td></tr> <tr> <td>Compression Software</td><td>✓</td><td></td></tr> </tbody> </table>	Programs	True	False	Disk Defragmenter	✓		Word Processor		✓	Library program		✓	Compression Software	✓		4
Programs	True	False															
Disk Defragmenter	✓																
Word Processor		✓															
Library program		✓															
Compression Software	✓																

71. M/J 18/P12/Q1/a,b

1(a)(i)	1 mark per bullet to max 3 • Storage space divided into file allocation units • Space allocated to particular files • Maintains/creates directory structures • Specifies the logical method of file storage (e.g. FAT or NTFS) • Provides file naming conventions • Controls access // implements access rights // implements password protection // Makes file sharing possible • Specifies tasks that can be performed on a file (e.g. open, close, delete, copy, create, move etc.)	3
---------	---	----------

1(a)(ii)	1 mark per bullet to max 3 • Installs printer driver • Sends data to the printer / buffer to print // sends documents to the print queue • Sends commands to printer • Receives and handles (error) messages/signals/interrupts from the printer	3															
1(b)(i)	1 mark for each correct box ticked. <table border="1"> <thead> <tr> <th>Program</th><th>True</th><th>False</th></tr> </thead> <tbody> <tr> <td>Database</td><td></td><td>✓</td></tr> <tr> <td>Virus checker</td><td>✓</td><td></td></tr> <tr> <td>Web browser</td><td></td><td>✓</td></tr> <tr> <td>Backup software</td><td>✓</td><td></td></tr> </tbody> </table>	Program	True	False	Database		✓	Virus checker	✓		Web browser		✓	Backup software	✓		4
Program	True	False															
Database		✓															
Virus checker	✓																
Web browser		✓															
Backup software	✓																
1(b)(ii)	1 mark for each valid utility program to max 2 e.g. • System clean up • Automatic update • Disk contents analysis / Disk checking / Disk repair • File compression • Disk formatter • Firewall • Disk Defragmenter	2															

72. M/J 18/P21/Q5/b

5(b)	Two from the following examples: • Context-sensitive help • Syntax checking (on entry) • Automatic indentation • Type checking (Parameter checking) • PrettyPrinting • Highlight structure blocks (e.g. selection, iteration) • Highlight any undeclared variables • Highlight any unassigned variables	Max 2
------	--	--------------

73. M/J 18/P22/Q5/b

5(b)	One mark for each of: • (Dynamic) syntax checking / Errors are highlighted / underlined • Type checking • Parameter checking • Identification of unused variables	Max 2
------	--	--------------

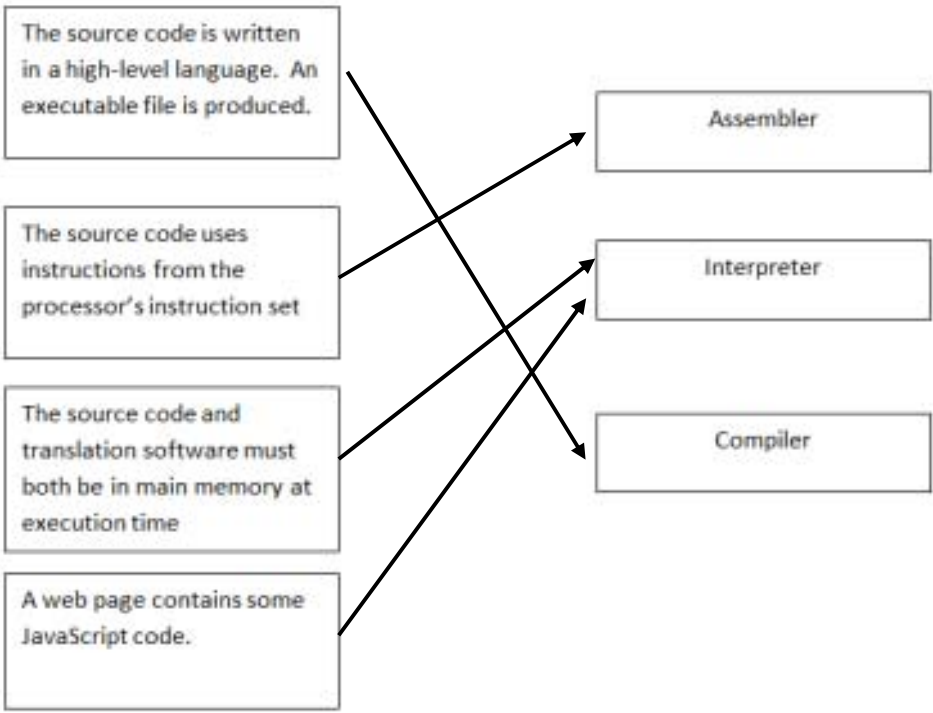
74. M/J 18/P23/Q5/b

5(b)	Two from these examples: • Indentation • Colour-coding of keywords /comments • Expansion / collapsing of complex data structures	Max 2
------	--	--------------

75. O/N 17/P12/Q1

1	1 Mark for stating the management task 1 Mark for a corresponding description Maximum 2 marks for each task Maximum 3 tasks Process / Task Management • Allocation of processor time • Scheduling of processes or tasks / multi-tasking / multi-programming etc. • By example – e.g. round-robin, shortest remaining time first etc. • Resolution of conflict when two or more processes require the same resource Secondary Storage management • Storage space divided into file allocation units • Space allocated to particular files • OS maintains a file directory and FAT • Provides file naming conventions • Controls access. Peripheral / Hardware / Device / Input/output Management • Installation of appropriate driver software • Controls access to data being sent to/from hardware/peripherals • Controls access to hardware/peripherals • Manages communication between devices / hardware and software Provision of a User interface • Allows user interaction with the computer system// Facilitates human computer communication • Hides the complexity of the hardware from the user • Or by example – e.g. GUI, command line etc. Interrupt Handling • Halts the execution of the current process • Stores the values of the current process on the stack • Loads and executes the appropriate ISR code • Use of priorities for handling simultaneous interrupts • Saves data on power outage Security Management • Makes provision for recovery when data is lost • Provides usernames and passwords / encryption / user accounts • Prevents unauthorised access • Ensures privacy of data Provision of a software platform / environment • On which other programs / applications can be run	6
---	--	----------

76. O/N 17/P12/Q2/a,b

2(a)	1 Mark for each correct connection  <pre> graph LR A["The source code is written in a high-level language. An executable file is produced."] --> C[Compiler] B["The source code uses instructions from the processor's instruction set"] --> A[Assembler] C["The source code and translation software must both be in main memory at execution time"] --> I[Interpreter] D["A web page contains some JavaScript code."] --> I </pre> The diagram shows four boxes on the left and three on the right. Arrows indicate the following connections: - Box 1 (top) to Compiler - Box 2 to Assembler - Box 3 to Interpreter - Box 4 to Interpreter	4
2(b)(i)	1 Mark per bullet, max 2 • Once translated the compiler software is not needed to run the program • Compiled code should execute faster • Compiler produces an executable file • The executable file produced by a compiler can be distributed without users having sight of the source code // source code is kept secure // users are unable to make changes to the program • Cross-compilation is possible	2
2(b)(ii)	1 Mark per bullet, max 2 • Easier de-bugging • The interpreter stops when error encountered • error can be corrected in real time • The interpreter translates a statement then executes it immediately • Parts of the program can be tested, without all the program code being available.	2

77. O/N 17/P13/Q2/a,b

2(a)	<pre> graph LR A[A web page contains a client-side script] --> I[Interpreter] B[Each instruction in the source code consists of an op code and an operand] --> I C[The source code is required at run-time] --> Co[Compiler] D[When the source code is translated, copies of the executable program can be distributed without the need for the source code] --> As[Assembler] </pre>	4
2(b)(i)	One mark from: - The program code can be translated to run on any processor / platform - Source code is translated into machine independent intermediate code not machine dependent code	1
2(b)(ii)	Two marks from: - Java uses a two-step translation process - Java code is partially interpreted – partially compiled - Code is translated first into intermediate code / "bytecode"... ...using the Java compiler - The bytecode is finally interpreted by the Java Virtual Machine	Max 2

78. O/N 17/P22/Q4/b

4(b)	- Single-stepping – to allow program statements to be executed one at a time - Breakpoints – to pause / stop the program at a specific line / statement - Variable / expression watch window – to monitor the value of variables / expressions as the program is run One mark for each Feature (text as above or equivalent) + 1 for meaningful explanation of use in context.	6
------	--	----------

4(a)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> PROCEDURE TestRandom (Repetitions : INTEGER) DECLARE Frequency : ARRAY [1 : 10] OF INTEGER DECLARE Expected : REAL / INTEGER //allow either DECLARE NextRandom : INTEGER DECLARE N : INTEGER FOR N ← 1 TO 10 Frequency[N] ← 0 ENDFOR Expected ← INT(Repetitions / 10) CALL RANDOMIZE() //Set random seed FOR N ← 1 TO Repetitions NextRandom ← INT(RND() * 10) + 1 Frequency[NextRandom] ← Frequency[NextRandom] + 1 ENDFOR OUTPUT "The expected frequency is " & Expected OUTPUT "Number Frequency Difference" FOR N ← 1 TO 10 OUTPUT N & " " & Frequency[N] & " " & Frequency[N] - Expected ENDFOR ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading (including parameter) 2. Array declaration – 10 or 11 elements 3. Array declaration – data type 4. Variable declaration for a loop counter (integer) or expected frequency (integer or real) 5. Variable declaration for next random value (For Python solutions, mark points 1 to 4 may be gained by suitable comments) 6. Initialise all elements of array 7. To set all elements to zero 8. Calculate expected frequency	16
------	--	----

Q4 (a): Visual Basic

```

Dim random As New Random()

Sub TestRandom(ByVal repetitions As Integer)
    Dim randinrange As Integer
    Dim i As Integer
    Dim num(1 To 10) As Integer
    Dim freq As Integer
    Dim difference As Integer

    freq = repetitions / 10    'calculate expected frequency

    For i = 1 To 10            'initialise array to store total frequencies
        num(i) = 0
    Next i

    For i = 1 To repetitions 'generate random numbers & increment
        appropriate freq
        randinrange = random.Next(1, 11)
        num(randinrange) = num(randinrange) + 1
    Next i

    Console.WriteLine("The expected frequency is " & freq)    'report header
    Console.WriteLine("Number    Frequency    Difference")    'column headers

    For i = 1 To 10    'calc & display difference between expected and actual
        freq
        difference = num(i) - freq
        Console.WriteLine(i & "        " & num(i) & "        " & difference)
    Next i

End Sub

```

Other possible ways of calculating a random number in VB include:

```

    randinrange = CInt(Math.Floor((upperbound - lowerbound + 1) * Rnd())) +
lowerbound

    randinrange = math.round((Rnd()*9)+1)

    randinrange = CInt(Math.Ceiling(Rnd() * 9

```

Q4 (a): Pascal

```
procedure TestRandom(var Repetitions : integer);
var
    Frequency : array[1..10] : integer;
    Expected, NextRandom, N : integer;

begin
    Expected := Round(Repetitions/10);
    for N := 1 to 10 do
        Frequency[N] := 0;

    for N := 1 to Repetitions do
        begin
            NextRandom := random(10)+1;
            Frequency[NextRandom] := Frequency[NextRandom]+1;
        end;

    writeln ('The expected frequency is ', Expected);
    writeln ('Number Frequency Difference');

    for N := 1 to 10 do
        writeln ('      ',N,'      ',Frequency[N],'      ',Frequency[N]-
Expected);

    end;
```

Q4 (a): Python

```

# frequency is an array from 1 to 10 of type integer;
# nextNumber is an integer which stores the created random number
# expected is an integer which stores the expected frequency of each number

def TestRandom (repetitions):
    import random
    frequency = [0 for i in range(1,11)] # initialise each frequency count
    to 0

    expected = repetitions / 10

    for i in range(1, repetitions + 1):
        nextNumber = random.randint(1,10)
        frequency[nextNumber] = frequency[nextNumber]+ 1

    print ("The expected frequency is ", expected)
    print("   Number   Frequency   Difference")

    for i in range(1,11):
        print ("      ", i, "      ", frequency[i], "      ", frequency[i] -
expected)

```

Alternative:

```

def TestRandom (repetitions):
    expected = repetitions / 10      ## initialised as real/integer
                                     ## NextRandom and N defined as integers
    frequency =[0,0,0,0,0,0,0,0,0,0,0] ## defined as an array and
    initialised to zero

    for n in range (0,repetitions):
        nextNumber = randint(1, 10)
        frequency[nextNumber] += 1

    print ('The expected frequency is ', expected)

    print ('Number   Frequency   Difference')
    for n in range (1, 11):
        print (n, '      ', frequency[n], '      ', frequency[n] - expected)

```

Alternative:

```

frequency =[0]*11      ## alternate way to initialise array to zero
frequency =[]          ## empty array/list

```

Alternative:

```

for n in range (1,11):
    frequency[n-1] = 0      ##alternate way to initialise array to zero

```

Alternative:

```
for n in range (0,11): ##alternate way to initialise array to zero
    frequency.append(0)
```

4(a)	9. Loop to generate required number of random values 10. Use of relevant <code>RANDOM()</code> function in a loop 11. Generate random integer value in the range 1 to 10 in a loop 12. Increment (array) element in a loop 13. Output expected frequency message not in any loop 14. Output column header text 15. (Loop to) output each row 16. ... including three correct values (spaces optional)	
------	--	--

79. M/J 17/P11/Q6/a,b

6(a)	One mark for each correct line from each left hand box to max three marks.	3
6(b)(i)	File compression software	1
6(b)(ii)	Backup software	1
6(b)(iii)	Disk repair software	1
6(b)(iv)	Anti-virus software	1

80. M/J 17/P12/Q4/a,b

4(a)(i)	Two from: - The hardware is unusable without an OS // hides complexity of hardware from user 1 - Acts as an interface/ controls communications between user and hardware / hardware and software // or by example 1 - Provides software <u>platform / environment</u> on which other programs can be run 1	2
---------	---	----------

4(a)(ii)	One mark for the name and one mark for description. Max two management tasks. - Provides the Human Computer Interface (HCI) 1 Controls communications between user and hardware// or by example 1 - Main memory management 1 Memory protection to ensure that two programs do not try to use the same space // Use of virtual memory // Location of processes within the memory // By example 1 - File / Secondary storage management 1 Maintains directory structures // Provides file naming conventions // Controls access 1 - Peripheral / hardware / device / Input-Output management 1 Installation of appropriate driver software // Controls access to data being sent to/from hardware/peripherals // Controls access to hardware/peripherals // manages communication between devices. 1 - Interrupt handling 1 Identifies priorities of interrupts // Saves data on power outage // Loads appropriate Interrupt Service Routine (ISR) // By example 1 - Security management 1 Makes provision for recovery when data is lost // Provides usernames and passwords // Prevents unauthorised access // Ensures privacy of data 1	Max 4
4(b)(i)	File compression software	1
4(b)(ii)	Backup software	1
4(b)(iii)	Disk defragmenting software	1
4(b)(iv)	Anti-virus software	1

81. M/J 17/P13/Q6/a,b

6(a)	One mark for each correct line from each left hand box to max <u>three</u> marks.	3
6(b)(i)	File compression software	1
6(b)(ii)	Backup software	1
6(b)(iii)	Disk repair software	1
6(b)(iv)	Anti-virus software	1

82. O/N 16/P11/Q2/b,c**(b) (i)** Any **two** from

- The hardware is unusable without an OS // hides complexity of hardware from user
- Acts as an interface / controls communications between user and hardware / hardware and software
- Provides software platform / environment on which other programs can be run [2]

(ii) Any **two** from:

- Process / task / resource management
- Main memory management
- Peripheral / hardware / device management
- File / secondary storage management
- Security management
- Provision of a software platform / environment on which other programs can be run
– only if not given in part (b)(i)
- Interrupt handling
- Provision of a user interface run – only if not given in part (b)(i) [2]

(c) Any **two** from:

- A DLL file is a shared library file
- Code is saved separately from the main .EXE files
- Code is only loaded into main memory when required at run-time
- The DDL file can be made available to several applications (at the same time) [2]

83. O/N 16/P11/Q7/a

7 (a) One mark for the name and **one mark** for the explanation for **three** utility programs

- Disk formatter
- Prepares a hard disk to allow data to be stored on it

- Virus checker
- Checks for viruses and then quarantines removes any virus found

- File compression
- Reduces file size by removing redundant details (lossy / lossless)

- Backup software
- Makes copy of files on another medium in case of corruption / loss of data

- Firewall
- Prevents unauthorised access to computer system from external sources [6]

84. O/N 16/P12/Q7

7 **ONE** mark per bullet point, **MAX TWO** marks per task.

- Process /resource management
- Scheduling of processes / multi-tasking / multi-programming etc.
- Resolution of conflicts when two or more processes require the same resource

- Main memory management
- Memory protection to ensure that two programs do not try to use the same space
- Use of virtual memory
- Deciding which processes need to be in main memory at any one time
- Location of processes within the memory
- By example, e.g. when process terminates, memory is made available

- Peripheral /hardware /device management
- Installation of appropriate driver software
- Controls access to data being sent to /from hardware /peripherals
- Controls access to hardware /peripherals
- Manages communication between devices /hardware and software

- File /secondary storage management
- Maintains directory structures
- Provides file naming conventions
- Controls access

- Security management
- Makes provision for recovery when data is lost
- Provides usernames and passwords /encryption /user accounts
- Prevents unauthorised access
- Ensures privacy of data

- Provision of a software platform / environment
- On which other programs can be run

- Interrupt handling
- Identifies priorities of interrupts
- Save current memory /process values / saves data on power outage
- Loads appropriate Interrupt Service Routine (ISR)
- Any relevant example

85. O/N 16/P12/Q8/a,b

8 (a) ONE mark for each bullet point from **MAX TWO** groups.

- The code is already written
- (So the programmer is not starting over again) which saves time
- The code will have been used by many people
- So it should be already thoroughly tested // relatively error-free
- The programmer can use, e.g. mathematical / graphics functions, etc. (may not know how to code)
- Can be sure that the function will perform as it should // simplifies the program.
- The code should conform to industry standards
- And therefore contribute towards a more robust program

[4]

(b) (i) ONE mark for each benefit, and **ONE** mark for a further expansion.

- The executable file is smaller / the executable does not contain all the library routines ...
- ... DLL files are only loaded into memory when required.
- Changes / improvements / error correction to the DLL file code are done independently of the main program...
- ... So there is no need to recompile the main program
- ... All programs using it will benefit
- A single DLL file can be made available to several application programs...
- ... Saving space in memory / easing the pressure on memory

[4]

(ii) **ONE** mark for each bullet point from **MAX ONE** group.

- The executable code is not self-contained ...
- ... the DLL file(s) needed to be included at run time.
- Appropriate (linking) software must be available at run-time ...
- ... to link/include/import the DLL files.
- The DLL file must be present ...
- ... otherwise (unable to find X.dll) errors
- Unexpected changes to the DLL file/corrupted DLL file ...
- ... could mean the program stops working as expected
- Malicious changes to the DLL file ...
- ... could install a virus on the user's computer/related files could be corrupted

[2]

86. O/N 16/P13/Q2/b,c

(b) (i) Any **two** from

- The hardware is unusable without an OS // hides complexity of hardware from user
- Acts as an interface / controls communications between user and hardware / hardware and software
- Provides software platform / environment on which other programs can be run [2]

(ii) Any **two** from:

- Process / task / resource management
- Main memory management
- Peripheral / hardware / device management
- File / secondary storage management
- Security management
- Provision of a software platform / environment on which other programs can be run – only if not given in part (b)(i)
- Interrupt handling
- Provision of a user interface run – only if not given in part (b)(i) [2]

(c) Any **two** from:

- A DLL file is a shared library file
- Code is saved separately from the main .EXE files
- Code is only loaded into main memory when required at run-time
- The DDL file can be made available to several applications (at the same time) [2]

87. O/N 16/P13/Q7/a

7 (a) One mark for the name and **one mark** for the explanation for **three** utility programs

- Disk formatter
- Prepares a hard disk to allow data to be stored on it
- Virus checker
- Checks for viruses and then quarantines removes any virus found
- File compression
- Reduces file size by removing redundant details (lossy / lossless)
- Backup software
- Makes copy of files on another medium in case of corruption / loss of data
- Firewall
- Prevents unauthorised access to computer system from external sources [6]

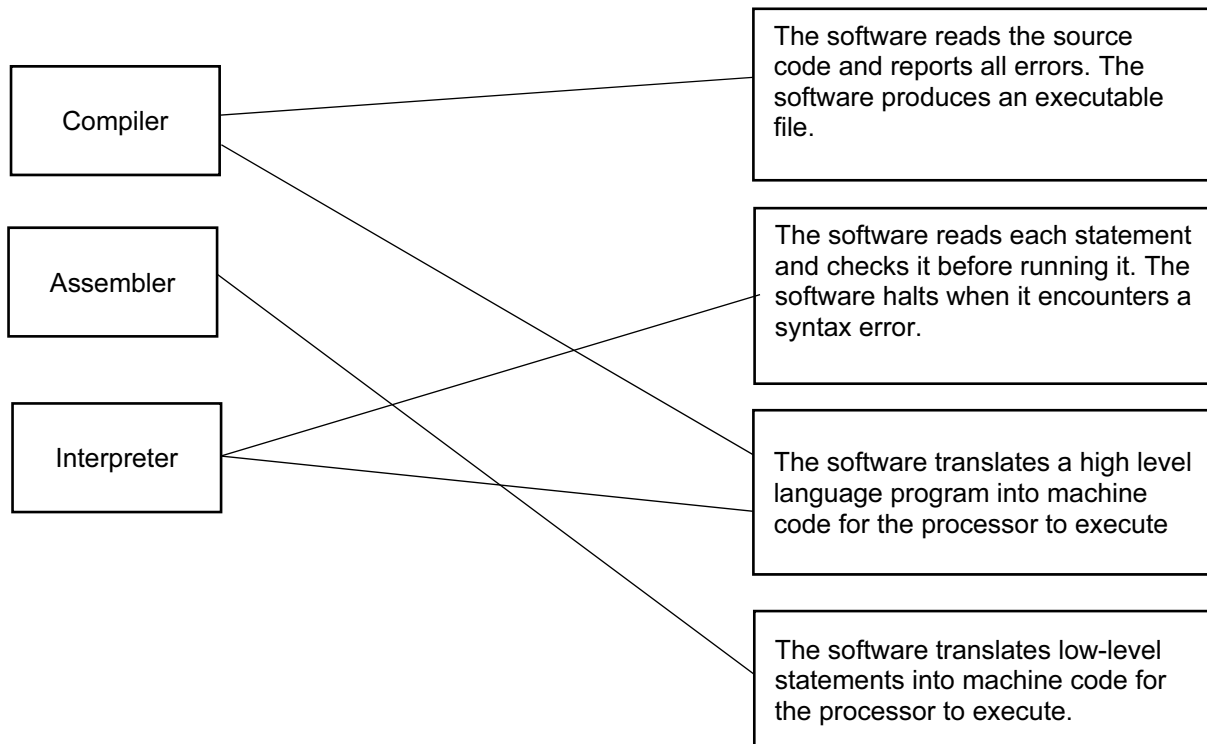
88. O/N 16/P21/Q4/a,b

- 4 (a)**
- Create / modify the source code using the text editor
 - Compiler translates the source code
 - Compiler produces the object code [Max 3]

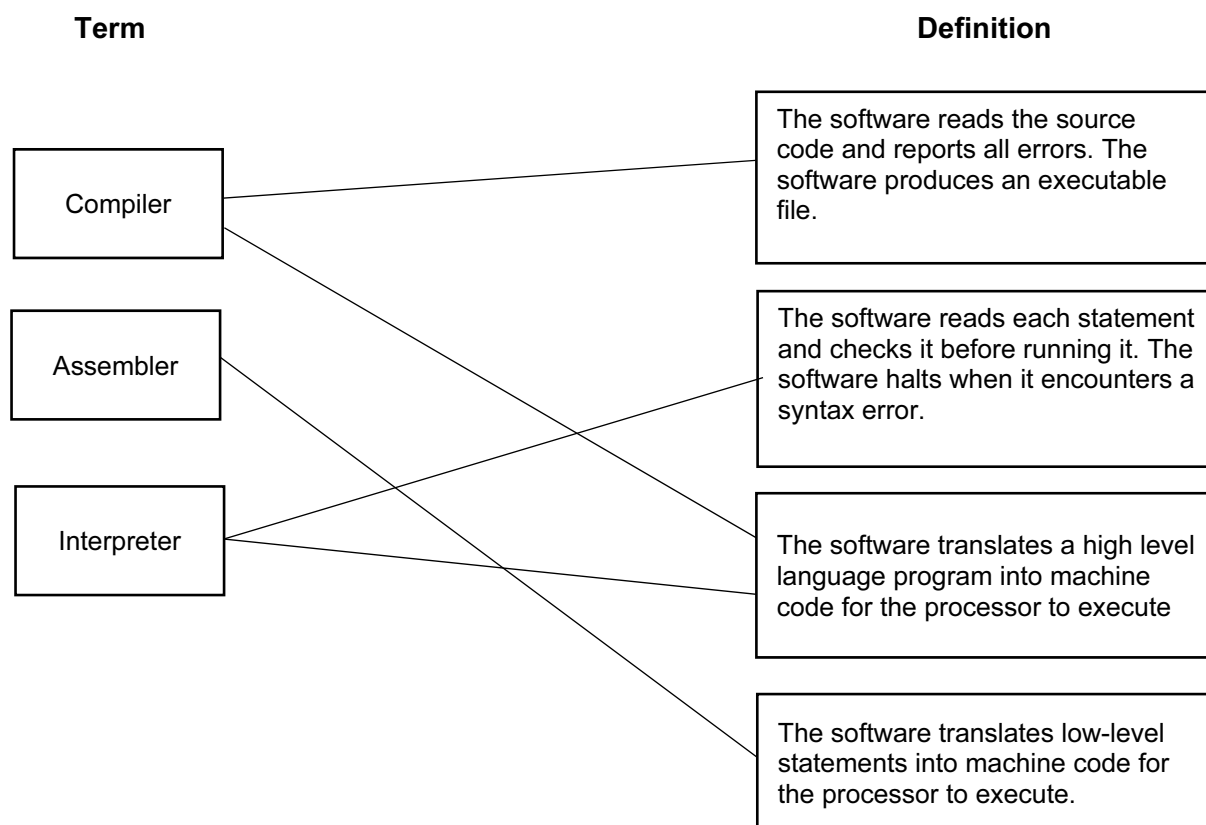
- (b) (i) • Errors in keywords are highlighted // before the compilation process
• Provides line-by-line syntax checking as code is typed in
• Provides line number of the error
• Display of known identifier names
• Auto-complete
• Colour-coding
• Auto-indent
• type checking
• Subroutine parameter checking [Max 1]
- (ii) • Set break-points
• Single step / step into/over subroutine
• Window to watch the changing value of variables [Max 1]

89. O/N 16/P23/Q4/a,b

- 4 (a) • Create / modify the source code using the text editor
• Compiler translates the source code
• Compiler produces the object code [Max 3]
- (b) (i) • Errors in keywords are highlighted // before the compilation process
• Provides line-by-line syntax checking as code is typed in
• Provides line number of the error
• Display of known identifier names
• Auto-complete
• Colour-coding
• Auto-indent
• type checking
• Subroutine parameter checking [Max 1]
- (ii) • Set break-points
• Single step / step into/over subroutine
• Window to watch the changing value of variables [Max 1]

90. M/J 16/P11/Q1**1 One mark** for each box on the left.**[3]****Term****Definition***Marks allocated as follows:*

- Compiler – **1 mark** for **two** correct connecting lines
Assembler – **1 mark** for **one** correct connecting line
Interpreter – **1 mark** for **two** correct connecting lines

91. M/J 16/P12/Q1**1 One mark** for each box on the left.**[3]***Marks allocated as follows:*

- Compiler – **1 mark** for **two** correct connecting lines
 Assembler – **1 mark** for **one** correct connecting line
 Interpreter – **1 mark** for **two** correct connecting lines

92. M/J 16/P13/Q1**1 Four** from:**[4]**

- Compiler creates an executable//an interpreter does not create an executable.
- The compiled program can be independently distributed.
- Compiler reports all errors at the end of compilation//an interpreter stops when it reaches an error.
- Interpreter executes each statement immediately after decoding/checking it//a compiler checks the whole program for errors.
- The interpreter software/source code must be present in main memory every time the program is executed//the compiled program does not require compiler/source code to be present.
- Cross-compilation is possible/compile on one hardware platform to run on another.

93. O/N 15/P11/Q11**11**

Statement	Interpreter	Compiler
This translator creates an executable file		✓
When this translator encounters a syntax error, game execution will halt	✓	
The translator analyses and checks each line just before executing it	✓	
This translator will produce faster execution of the game program		✓
Use of this translator makes it more difficult for the user to modify the code of the game supplied to the user		✓

1 mark for each correct row

[5]

94. O/N 15/P13/Q11**11**

Statement	Interpreter	Compiler
This translator creates an executable file		✓
When this translator encounters a syntax error, game execution will halt	✓	
The translator analyses and checks each line just before executing it	✓	
This translator will produce faster execution of the game program		✓
Use of this translator makes it more difficult for the user to modify the code of the game supplied to the user		✓

1 mark for each correct row

[5]

95. O/N 15/P21/Q2

- 2 (i) (Single) software program [1]
Features for:
program editor/writing/editing
translation // interpreter/compiler
testing program code // observe outputs } 2 points to score [1]
- (ii) Syntax checking (on entry)
Structure blocks (e.g. IF structure and loops begin/end highlighted)
General prettyprint features
Automatic indentation
Highlights any undeclared variables
Highlights any unassigned variables
Commenting out/in of blocks of code
Visual collapsing / highlighting of blocks of code
Single stepping
Breakpoints
Variable/expressions report window [MAX 3]

96. O/N 15/P23/Q2

- 2 (i) (Single) software program [1]
Features for:
program editor/writing/editing
translation // interpreter/compiler
testing program code // observe outputs } 2 points to score [1]

- (ii) Syntax checking (on entry)
Structure blocks (e.g. IF structure and loops begin/end highlighted)
General prettyprint features
Automatic indentation
Highlights any undeclared variables
Highlights any unassigned variables
Commenting out/in of blocks of code
Visual collapsing / highlighting of blocks of code
Single stepping
Breakpoints
Variable/expressions report window

[MAX 3]